

and generator matrix

$$G = \begin{bmatrix} 4 & 4 & 1 & 0 & 0 & 0 \\ 4 & 3 & 0 & 1 & 0 & 0 \\ 4 & 2 & 0 & 0 & 1 & 0 \\ 4 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

2. The binary repetition code of length n is a cyclic code with generator polynomial $1 + x + \cdots + x^{n-1}$. The binary even weight code is a cyclic code with generator polynomial $1 + x$.
3. Over $\mathbb{F}_2$, the polynomial $x^7 - 1 = x^7 + 1 = (1 + x)(1 + x + x^3)(1 + x^2 + x^3)$. Hence, $g(x) = 1 + x + x^3$ is the generator polynomial of a binary $[7, 4]$ -cyclic code. The corresponding parity check polynomial is $h(x) = (1 + x)(1 + x^2 + x^3) = 1 + x + x^2 + x^4$. So, the generator matrix G and parity check matrix H for this code are given by

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Noting that the columns of H represent all the nonzero elements of $\mathbb{F}_2^3$, we see that this code is an equivalent version of the Hamming $[7, 4]$ -perfect code.

61.5 Important Classes of Linear Codes 2

Definitions:

The **Golay codes** [Gol49] are two cyclic codes defined as follows.

The **binary Golay code**, G_{23} , is a $[23, 12, 7]$ binary cyclic code with generator polynomial $g(x) = (x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1)$.

The **ternary Golay code**, G_{11} , is an $[11, 6, 5]$ cyclic code over the ternary field $\mathbb{F}_3$ with generator polynomial $g(x) = (x^5 + 2x^3 + x^2 + 2x + 2)$.

Denote by $\mathbb{F}[x; k-1]$ the set of all polynomials of degree at most $k-1$ over $\mathbb{F}$. Assume $\mathbb{F}$ has size $|\mathbb{F}| > n$ and let α be a primitive of $\mathbb{F}$ (i.e., α is a generator of the cyclic group $\mathbb{F}^*$). Define the linear map (called the evaluation map) by

$$\begin{aligned} ev : \mathbb{F}[x; k-1] &\longrightarrow \mathbb{F}^n \\ f &\longmapsto (f(\alpha), \dots, f(\alpha^n)). \end{aligned}$$

The image of this map is a **Reed–Solomon code**.

Let $\mathbb{F} = \mathbb{F}_q$ be a fixed finite field and consider the polynomial $p(x) = x^n - 1 \in \mathbb{F}[x]$ and let the root w of $p(x)$ be a primitive n th root of unity. Let b, d be positive integers.

$$\mathcal{B}_q(n, d, b, w) := \{c(x) \in \mathbb{F}[x; k-1] \mid c(w^{b+i}) = 0 \text{ for } i = 0, \dots, d-2\}$$

is called a **Bose–Chadhuri–Hocquenghem (BCH) code** having designed distance d . In the special case when $n = q^m - 1$, $\mathcal{B}_q(n, d, b, w)$ is a **primitive BCH code**; and if $b = 1$, $\mathcal{B}_q(n, d, b, w)$ is a **narrow sense BCH code**.

Facts:

1. Over the binary field $\mathbb{F}_2$,

$$x^{23} + 1 = (x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1)(x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1)(x + 1).$$

Therefore, the binary Golay code, G_{23} , has parity check polynomial $h(x) = (x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1)(x + 1)$.

2. [Hil86, pp. 153–157] The binary Golay code G_{23} is a $[23, 12, 7]$ code having 4096 elements. G_{23} is a perfect code.
3. Over the ternary field $\mathbb{F}_3$,

$$x^{11} - 1 = (x^5 + 2x^3 + x^2 + 2x + 2)(x^5 + x^4 + 2x^3 + x^2 + 2)(x + 2).$$

Therefore, the ternary Golay code, G_{11} , has parity check polynomial $h(x) = (x^5 + x^4 + 2x^3 + x^2 + 2)(x + 2)$.

4. [Hil86, pp. 157–159] The Golay code G_{11} is a ternary $[11, 6, 5]$ code having 729 elements. G_{11} is a perfect code.
5. [Hil86, Theorem 9.5] Here is a complete catalog of perfect linear codes up to equivalence:
 - (a) The trivial perfect codes: All of $\mathbb{F}^n$, and the binary repetition codes of odd length.
 - (b) The Hamming codes.
 - (c) The Golay codes G_{23} and G_{11} .

6. Regarding Reed–Solomon codes, the linear map $e\nu$ is one-to-one. Hence, the defined Reed–Solomon code is an $[n, k]$ code. By the fundamental theorem of algebra the minimum weight of a nonzero code word is $n - k + 1$. It follows that a Reed–Solomon code is MDS. With regard to the natural basis $1, x, \dots, x^{k-1}$ of $\mathbb{F}[x; k - 1]$, this code has a generator matrix:

$$G := \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha & \alpha^2 & \dots & \alpha^n \\ \vdots & & & \vdots \\ \alpha^{k-1} & \alpha^{2(k-1)} & \dots & \alpha^{n(k-1)} \end{bmatrix}.$$

7. Regarding BCH codes, it is immediate from the definition that $\mathcal{B}_q(n, d, b, w)$ has a parity check matrix of the form

$$H := \begin{bmatrix} 1 & w^b & \dots & w^{(n-1)b} \\ 1 & w^{b+1} & \dots & w^{(n-1)(b+1)} \\ \vdots & & & \vdots \\ 1 & w^{b+d-2} & \dots & w^{(n-1)(b+d-2)} \end{bmatrix}.$$

We would like to stress that $\mathcal{B}_q(n, d, b, w)$ is the $\mathbb{F}_q$ kernel of H and that the entries of H are in general not in the base field $\mathbb{F}_q$. Assume $w \in \mathbb{F}_{q^s}$, where $\mathbb{F}_{q^s}$ is an extension field of $\mathbb{F}_q$. The $\mathbb{F}_{q^s}$ kernel of H is a Reed–Solomon code of distance d . As a consequence, we have that $\mathcal{B}_q(n, d, b, w)$ has distance at least d .

8. [Rom92, Chap. 8] The BCH code $\mathcal{B}_q(n, d, b, w)$ is a cyclic code. For $i = 0, \dots, d - 2$, let $m_i(x) \in \mathbb{F}_q[x]$ be the minimal polynomial of w^{b+i} and let

$$g(x) := \text{lcm}\{m_i(x) \mid i = 0, \dots, d - 2\} \in \mathbb{F}_q[x].$$

Then $g(x)$ is a generator of the cyclic code $\mathcal{B}_q(n, d, b, w)$ and its dimension is

$$\dim \mathcal{B}_q(n, d, b, w) = n - \deg g(x).$$

9. [Mas69], [FWRT94], [Sud97], [KV03] Reed–Solomon codes and BCH codes can be efficiently decoded up to half the designed distance using the Berlekamp–Massey algorithm [Mas69]. Extensions for this algorithm exist for algebraic geometric codes [FWRT94]. These algorithms have the major drawback that no **soft information** can be processed. This means that the decoder needs a vector in $\mathbb{F}^n$ as an input and it is not possible to process real numbers as they naturally occur in wireless communication. In 1997 Sudan [Sud97] achieved a major breakthrough when he came up with a polynomial time algorithm for list decoding of Reed–Solomon codes. Since that time the technique has been vastly improved and generalized and a broad overview is given in [Gur04] and [KV03].

Examples:

1. *Algebraic Geometric Codes.* In 1977, V.D. Goppa had the idea to vastly generalize the construction techniques of Reed–Solomon codes and BCH codes. Goppa codes became famous in 1982 when Tsfasman, Vladut, and Zink [TVZ82] constructed an infinite family of codes that exceeds the Gilbert Varshamov bound. (For further details, the reader is referred to the extensive literature on this subject [Gop81], [Gop88], [LG88], [TV91], and [Wal00].

Consider the **projective plane** $\mathbb{P}_{\mathbb{F}}^2$ and let $V \subset \mathbb{P}_{\mathbb{F}}^2$ be a **smooth curve** of degree θ . This means there is an irreducible homogeneous polynomial $\varphi(x, y, z) \in \mathbb{F}[x, y, z]$ of degree θ with

$$V = \{(\alpha, \beta, \gamma) \in \mathbb{P}_{\mathbb{F}}^2 \mid \varphi(\alpha, \beta, \gamma) = 0\}.$$

The smoothness is characterized by the fact that there is no point on the curve where the partial derivatives vanish simultaneously. $\Gamma(V) := \mathbb{F}[x, y, z] / \langle \varphi \rangle$ is called the homogeneous coordinate ring. As $\Gamma(V)$ is a domain, there is a well-defined quotient field $k(V)$, which is called the **homogeneous function field**. (See e.g. [Ful89, p.91].)

If $f \in k(V)$, then f has an **associated divisor**

$$(f) := \sum_{P \in V} \text{ord}_P(f) P,$$

where $\text{ord}_P(f)$ is m if f has a pole of order m at P and $-m$ if f has a zero of order m at P .

Let $D = P_1 + \cdots + P_n$ be a divisor on V of degree n . Define the vector space

$$L(D) := \{f \in k(V)^* \mid (f) + D \geq 0\} \cup \{0\}.$$

Let G be a divisor on V having support disjoint from D . Consider the linear map:

$$\begin{aligned} ev : L(G) &\longrightarrow \mathbb{F}^n \\ f &\longmapsto (f(P_1), \dots, f(P_n)). \end{aligned}$$

The image $\mathcal{C}(D, G)$ of the linear evaluation map ev is called an **algebraic geometric Goppa (AG) code**. Note that $\mathbb{F}[x; k-1]$ is a linear space of the form $L(G)$ and in this way Reed–Solomon codes are AG codes.

Let $\mathcal{C}(D, G)$ be an algebraic geometric Goppa code defined from a curve of degree θ . If $\deg G < n$, then

$$\dim \mathcal{C}(D, G) = \dim L(G) - \dim L(G - D) \geq \deg G + 1 - \frac{1}{2}(\theta - 1)(\theta - 2)$$

and for the distance one has the lower bound

$$d(\mathcal{C}(D, G)) \geq n - \deg G.$$

Note that $\frac{1}{2}(\theta - 1)(\theta - 2)$ is the genus of the curve. One way to construct AG codes with good distance parameters is, therefore, to construct curves of low genus having many $\mathbb{F}_q$ rational points.

2. *Low density parity check codes.* A class of linear codes of tremendous practical importance was introduced by R.G. Gallager in his dissertation [Gal63]. These binary linear codes were called **low density parity check (LDPC) codes** by the author and are characterized by the property that the code can be written as the kernel of a very sparse matrix. For example, Gallager studied codes whose parity check matrix had three 1s in every column and six 1s in every row randomly chosen.

LDPC codes come with the remarkable property that efficient decoding algorithms exist whose complexity increases linearly with the code length [RU01]. One of simplest algorithms is the **bit flipping algorithm**. For this, assume that H is a sparse parity check matrix, a code word $\mathbf{v}$ was sent, and a vector $\mathbf{y} = \mathbf{v} + \mathbf{e}$ was received. The algorithm then checks bit-by-bit if the weight of the syndrome vector $\mathbf{y}H^T$ increases or decreases if the bit under consideration is flipped. The bit flipping algorithm is one of the most simple type of message passing algorithms.

LDPC codes were later generalized by R. M. Tanner [Tan81] who defined linear codes defined through graphs. A good overview to this active area of research can be found in [MR01]. The class of LDPC codes and codes on graphs became prominent again after the discovery of Turbo codes [BGT93].

61.6 Convolutional Codes

Aside from linear block codes, the codes most used in engineering practice are convolutional codes. For this, consider the situation where a large amount of data has to be encoded by a linear block code. Let $\mathbf{m}_0, \dots, \mathbf{m}_\tau \in \mathbb{F}^k$ be $\tau + 1$ blocks of messages to be transmitted and assume G is the generator matrix of a linear $[n, k]$ code. The encoding scheme:

$$\mathbf{m}_i \mapsto \mathbf{m}_i G = \mathbf{x}_i, \quad i = 0, \dots, \tau$$

can be compactly written in the form

$$\mathbf{m}(z) \mapsto \mathbf{m}(z)G = \mathbf{x}(z),$$

where

$$\mathbf{m}(z) := \sum_{i=0}^{\tau} \mathbf{m}_i z^i \in \mathbb{F}^k[z] \quad \text{and} \quad \mathbf{x}(z) := \sum_{i=0}^{\tau} \mathbf{x}_i z^i \in \mathbb{F}^n[z].$$

Elias [Eli55] had the original idea to replace in this encoding scheme the generator matrix G with a $k \times n$ matrix $G(z)$ whose entries are elements of the polynomial ring $\mathbb{F}[z]$.

Definitions:

Consider the polynomial ring $R := \mathbb{F}[z]$. A submodule $\mathcal{C} \subseteq R^n$ is called a **convolutional code**. The **rank** of the code $\mathcal{C}$ is the rank, k , of $\mathcal{C}$ considered as an R -module. $\mathcal{C}$ is an $[n, k]$ -convolutional code. The **rate** of $\mathcal{C}$ is k/n .

A **generator matrix** for the rate k/n convolutional code $\mathcal{C}$ is a $k \times n$ polynomial matrix G with entries in R such that

$$\mathcal{C} = \{\mathbf{m}G = \mathbf{x} \mid \mathbf{m} \in R^k, \mathbf{x} \in R^n\}.$$

The **degree** of the convolutional code $\mathcal{C}$ with generator matrix G is defined to be the largest degree of any $k \times k$ minor of G .

Let $\mathbf{x}(z) = \sum_{i=0}^{\tau} \mathbf{x}_i z^i \in R^n$ be a code vector. Define the **weight of $\mathbf{x}(z)$** by

$$\text{wt}(\mathbf{x}(z)) := \sum_{i=0}^{\tau} \text{wt}(\mathbf{x}_i).$$

The **free distance** of the convolutional code $\mathcal{C}$ is defined as

$$d_{\text{free}}(\mathcal{C}) := \min\{\text{wt}(\mathbf{x}(z)) \mid \mathbf{0} \neq \mathbf{x}(z) \in \mathcal{C}\}. \quad (61.1)$$

Facts:

1. [Hun74] R is a principal ideal domain and $\mathcal{C}$ is, therefore, a free R -module. As such, $\mathcal{C}$ has a well-defined rank k , and a $k \times n$ generator matrix G with entries in R , such that

$$\mathcal{C} = \{\mathbf{m}G = \mathbf{x} \mid \mathbf{m} \in R^k, \mathbf{x} \in R^n\}.$$

2. If G_1 and G_2 are both $k \times n$ generator matrices of the same convolutional code $\mathcal{C}$, then there exists a unimodular matrix $U \in GL_k(R)$ such that $G_2 = UG_1$.
3. The degree of a convolutional code is well-defined (i.e., it does not depend on the particular generator matrix chosen). In the literature the degree is sometimes also called the **total memory** [LC83] or the **overall constraint length** [JZ99] or the **complexity** [Pir88].
4. The free distance of a convolutional code measures the smallest distance between any two different code words.
5. A major design problem in the area of convolutional codes is the construction of $[n, k]$ codes of degree δ whose free distance is maximal or near maximal. This question constitutes the generalization of the **main linear coding problem**.
6. If $\mathcal{C}$ is an $[n, k]$ convolutional code having $G(z)$ as a generator matrix, then there exists an $(n - k) \times n$ parity check matrix $H(z)$ with entries in R . $H(z)$ is characterized by the property that $G(z)H(z)^T = \mathbf{0}_{k \times k}$.
7. Convolutional codes of degree zero correspond to linear block codes.
8. Convolutional codes of small degree can be efficiently decoded using the Viterbi decoding algorithm [LC83, Chap. 11]. This algorithm has the distinct advantage that soft decision can be processed and due to this fact convolutional codes became the codes of choice for many wireless applications.
9. The free distance of an $[n, k]$ convolutional code of degree δ must satisfy the **generalized Singleton bound** [RS99]:

$$d_{\text{free}}(\mathcal{C}) \leq (n - k) \left(\left\lceil \frac{\delta}{k} \right\rceil + 1 \right) + \delta + 1.$$

10. Convolutional codes achieving the generalized Singleton bound are called **MDS convolutional codes**. Such codes exist as soon as the field size is large enough [RS99].
11. The literature on convolutional codes is not consistent in terms of terminology and definitions. Classical articles are [MS67] and [For70]. The articles [McE98] and [Ros01] provide surveys on recent developments and further connections. Generalizations to multidimensional convolutional codes were considered and the reader will find further reading and references in [GLRW00].

In contrast to the theory of linear block codes, there are not so many algebraic construction techniques of convolutional codes with a good designed distance and even fewer constructions of codes that can be efficiently decoded. The reader will find algebraic constructions of convolutional codes in [GLS04], [Jus75], [Pir88], [RSY96], [SGLR01], and in the bibliography of these articles.

Examples:

1. Consider the finite field $\mathbb{F}_5$. The generator matrix $G(z) = (z, z + 1, z + 2, z + 3)$ defines an $[n, k] = [4, 1]$ convolutional code $\mathcal{C}$ of degree $\delta = 1$ and distance $d_{\text{free}}(\mathcal{C}) = 7$. The generalized Singleton bound for these parameters n, k , and δ is 8 and the defined code is, therefore, not an MDS convolutional code.

2. [RS99, Ex. 2.13]. Consider the finite field $\mathbb{F}_7$ and the generator matrix

$$G(z) = \begin{bmatrix} (z^2 + 1) & (3z^2 + 1) & (5z^2 + 1) \\ (z - 1) & (z - 2) & (2z - 3) \end{bmatrix}.$$

Then $G(z)$ defines an $[n, k] = [3, 2]$ convolutional code $\mathcal{C}$ of degree $\delta = 3$ and distance $d_{\text{free}}(\mathcal{C}) = 6$. The generalized Singleton bound for these parameters n, k , and δ is 6 and the defined code is therefore an MDS convolutional code.

References

- [BGT93] C. Berrou, A. Glavieux, and P. Thitimajshima. Near Shannon limit error-correcting coding and decoding: Turbo-codes. In *Proc. of IEEE Int. Conference on Communication*, pp. 1064–1070, Geneva, Switzerland, May 1993.
- [Bla03] R.E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.
- [Eli55] P. Elias. Coding for noisy channels. *IRE Conv. Rec.*, 4:37–46, 1955.
- [For70] G.D. Forney, Jr. Convolutional codes I: Algebraic structure. *IEEE Trans. Inform. Theory*, IT-16(5):720–738, 1970.
- [Ful89] W. Fulton. *Algebraic Curves*. Addison Wesley, Reading, MA, 1989. (Originally published by Benjamin/Cummings in 1969.)
- [FWRT94] G.L. Feng, V.K. Wei, T.R.N. Rao, and K. Tzeng. Simplified understanding and efficient decoding of algebraic geometric codes. *IEEE Trans. Inform. Theory*, IT-40(4):981–1002, 1994.
- [Gal63] R.G. Gallager. *Low-Density Parity Check Codes*. M.I.T. Press, Cambridge, MA, 1963. Number 21 in Research monograph series.
- [GLRW00] H. Gluesing-Luerssen, J. Rosenthal, and P. A. Weiner. Duality between multidimensional convolutional codes and systems. In F. Colonius, U. Helmke, F. Wirth, and D. Prätzel-Wolters, Eds., *Advances in Mathematical Systems Theory, A Volume in Honor of Diederich Hinrichsen*, pp. 135–150. Birkhauser, Boston, 2000.
- [GLS04] H. Gluesing-Luerssen and W. Schmale. On cyclic convolutional codes. *Acta Appl. Math*, 82:183–237, 2004.
- [Gol49] M.J.E. Golay. Notes on digital coding. *Proc. IEEE*, 37:657, 1949.
- [Gop81] V.D. Goppa. Codes on algebraic curves. *Soviet Math. Dokl.*, 24(1):170–172, 1981.
- [Gop88] V.D. Goppa. *Geometry and Codes*. Kluwer Academic Publisher, Dordrecht, The Netherlands, 1988.
- [Gur04] V. Guruswami. *List Decoding of Error-Correcting Codes*, Vol. 3282 of *Lecture Notes in Computer Science*. Springer, Heidelberg, 2004.
- [Hil86] R. Hill. *A First Course in Coding Theory*. Oxford Applied Mathematics and Computing Science Series. The Clarendon Press/Oxford University Press, New York, 1986.
- [HP03] W.C. Huffman and V. Pless. *Fundamentals of Error-Correcting Codes*. Cambridge University Press, Cambridge, 2003.
- [Hun74] T.W. Hungerford. *Algebra*. Graduate Texts in Mathematics. Springer, New York, 1974.
- [Jus75] J. Justesen. An algebraic construction of rate $1/\nu$ convolutional codes. *IEEE Trans. Inform. Theory*, IT-21(1):577–580, 1975.
- [JZ99] R. Johannesson and K.Sh. Zigangirov. *Fundamentals of Convolutional Coding*. IEEE Press, New York, 1999.
- [KV03] R. Koetter and A. Vardy. Algebraic soft-decision decoding of Reed–Solomon codes. *IEEE Trans. Inform. Theory*, 49(11):2809–2825, 2003.
- [LC83] S. Lin and D.J. Costello, Jr. *Error Control Coding: Fundamentals and Applications*. Prentice-Hall, Upper Saddle River, NJ, 1983.

- [LG88] J.H. van Lint and G. van der Geer. *Introduction to Coding Theory and Algebraic Geometry*. Birkhäuser Verlag, Basel, 1988.
- [Lin82] J.H. van Lint. *Introduction to Coding Theory*. Springer-Verlag, Berlin, New York, 1982.
- [Mas69] J.L. Massey. Shift-register synthesis and BCH decoding. *IEEE Trans. Inform. Theory*, IT-15:122–127, 1969.
- [McE98] R.J. McEliece. The algebraic theory of convolutional codes. In V. Pless and W.C. Huffman, Eds., *Handbook of Coding Theory*, Vol. 1, pp. 1065–1138. Elsevier Science Publishers, Amsterdam, The Netherlands, 1998.
- [MR01] B. Marcus and J. Rosenthal, Eds. *Codes, Systems and Graphical Models*. IMA Vol. 123. Springer-Verlag, New York, 2001.
- [MS67] J.L. Massey and M.K. Sain. Codes, automata, and continuous systems: explicit interconnections. *IEEE Trans. Automat. Contr.*, AC-12(6):644–650, 1967.
- [MS77] F.J. MacWilliams and N.J.A. Sloane. *The Theory of Error-Correcting Codes*. North Holland, Amsterdam, 1977.
- [PHB98] V.S. Pless, W.C. Huffman, and R.A. Brualdi, Eds. *Handbook of Coding Theory. Vol. I, II*. North-Holland, Amsterdam, 1998.
- [Pir88] Ph. Piret. *Convolutional Codes, an Algebraic Approach*. MIT Press, Cambridge, MA, 1988.
- [Rom92] S. Roman. *Coding and Information Theory*. Graduate Texts in Mathematics. Springer-Verlag, New York/Berlin, 1992.
- [Ros01] J. Rosenthal. Connections between linear systems and convolutional codes. In B. Marcus and J. Rosenthal, Eds., *Codes, Systems and Graphical Models*, IMA Vol. 123, pp. 39–66. Springer-Verlag, Heidelberg, 2001.
- [RS99] J. Rosenthal and R. Smarandache. Maximum distance separable convolutional codes. *Appl. Alg. Eng. Comm. Comp.*, 10(1):15–32, 1999.
- [RSY96] J. Rosenthal, J.M. Schumacher, and E.V. York. On behaviors and convolutional codes. *IEEE Trans. Inform. Theory*, 42(6, part 1):1881–1891, 1996.
- [RU01] T. Richardson and R. Urbanke. An introduction to the analysis of iterative coding systems. In B. Marcus and J. Rosenthal, Eds., *Codes, Systems and Graphical Models*, IMA Vol. 123, pp. 1–37. Springer-Verlag, Heidelberg, 2001.
- [SGLR01] R. Smarandache, H. Gluesing-Luerssen, and J. Rosenthal. Constructions for MDS-convolutional codes. *IEEE Trans. Inform. Theory*, 47(5):2045–2049, 2001.
- [Sud97] M. Sudan. Decoding of Reed–Solomon codes beyond the error-correction bound. *J. Complex.*, 13(1):180–193, 1997.
- [Tan81] R.M. Tanner. A recursive approach to low complexity codes. *IEEE Trans. Inform. Theory*, 27(5):533–547, 1981.
- [TV91] M.A. Tsfasman and S.G. Vlăduț. *Algebraic Geometric Codes*. Mathematics and Its Application. Kluwer Academic Publishers, Dordrecht, The Netherlands, 1991.
- [TVZ82] M.A. Tsfasman, S.G. Vlăduț, and Th. Zink. On Goppa codes which are better than the Varshamov–Gilbert bound. *Math. Nachr.*, 109:21–28, 1982.
- [Wal00] J.L. Walker. *Codes and Curves*. American Mathematical Society, Providence, RI, 2000. IAS/Park City Mathematical Subseries.

Web Resources

1. <http://www.win.tue.nl/~aeb/voorlincod.html> (A.E. Brouwer’s Web site on bounds on the minimum distance of linear codes).
2. <http://www.eng.tau.ac.il/~litsyn/tableand/> (Simon Litsyn’s table of nonlinear binary codes).
3. <http://www.research.att.com/~njas/codes/> (Neil J. A. Sloane’s Web site of linear [and some non-linear] codes over various fields).

62

Quantum Computation

Zijian Diao
Ohio University Eastern

62.1	Basic Concepts	62-2
62.2	Universal Quantum Gates	62-7
62.3	Deutsch's Problem	62-8
62.4	Deutsch–Jozsa Problem	62-9
62.5	Bernstein–Vazirani Problem	62-11
62.6	Simon's Problem	62-13
62.7	Grover's Search Algorithm.....	62-15
62.8	Shor's Factorization Algorithm	62-17
	References	62-19

Modern computer science emerged when the eminent British mathematician Alan Turing invented the concept of Turing machine (TM) in 1936. Though very simple and primitive, TM serves as the universal model for all known physical computation devices. The principles of quantum mechanics, another revolutionary scientific discovery of the 20th century, had never been incorporated in the theory of computation until the early 1980s. P. Benioff first coined the concept of quantum Turing machine (QTM). Motivated by the problem that classical computers cannot simulate quantum systems efficiently, R. Feynman posed the quantum computer as the solution. The field of quantum computation was born.

Quantum computation mainly studies the construction and analysis of quantum algorithms that outperform the classical counterparts. In terms of computability, quantum computers and classical computers possess exactly the same computational power. But in terms of computational complexity, which measures the efficiency of computation, there are many examples confirming that quantum computers do solve certain problems faster. The two most significant ones are Shor's factorization algorithm and Grover's search algorithm, among other examples such as the Deutsch–Jozsa problem, the Bernstein–Vazirani problem, and Simon's problem.

Quantum computers share many common features of the classical computers. In a classical computer, information is encoded in binary states (for example, 0 denotes the low voltage state and 1 denotes the high voltage state), and processed by various logic gates. In a quantum computer, information is represented by the states of the microscopic quantum systems, called qubits, and manipulated by various quantum gates. A qubit could be a two-level atom in the excited/ground states, a photon with horizontal/vertical polarizations, or a spin- $\frac{1}{2}$ particle with up/down spins. The state of a qubit can be controlled via physical devices such as laser and microwave. The distinctions between quantum and classical computers originate from the special characteristic of quantum mechanics. In contrast to a classical system, a quantum system can exist in different states at the same time, an interesting phenomenon called *superposition*. Superposition enables quantum computers to process data in parallel. That is why a quantum computer can solve certain

problems faster than a classical computer. But, when we measure a quantum system, it randomly collapses to one of the basis states. This indeterministic nature makes the design of efficient quantum algorithms highly nontrivial. Another distinctive feature of the quantum computer is that the operations performed by quantum gates must be unitary. This is the natural consequence of the unobserved quantum systems evolving according to the Schrödinger equation.

Throughout this chapter, we will use Dirac's bra-ket notation (see Section 59.4 for more information). In quantum mechanics, the state of a quantum system is described by a unit vector in a complex Hilbert space (a complete inner product vector space). Under Dirac's bra-ket notation, we use $|\psi\rangle$ ("ket") to denote a vector in the Hilbert space and $\langle\psi|$ ("bra") for its dual. The inner product of two vectors $|\psi\rangle$ and $|\phi\rangle$ is denoted $\langle\psi|\phi\rangle$. We also use $|\psi\rangle\langle\phi|$ and $|\psi\phi\rangle$ interchangeably with the notation for the tensor product $|\psi\rangle \otimes |\phi\rangle$.

62.1 Basic Concepts

Definitions:

A **(classical) Turing machine** (TM) is an abstract computing device consisting of a finite control, a two-way infinite tape, and a read/write head that moves to the left or right on the tape. It can be described by a 6-tuple $(Q, A, B, \delta, q_0, q_a)$, where Q is a finite set of control states, A a finite alphabet, $B \in A$ the blank symbol, $q_0, q_a \in Q$ the initial and accepting states, and δ the transition function

$$\delta : Q \times A \rightarrow Q \times A \times \{L, R\}.$$

L and R stand for moving left and right, respectively.

A **quantum Turing machine** (QTM) is an abstract computing device consisting of a finite control, a two-way infinite tape, and a read/write head that moves to the left or right of the tape. It can be described by a 6-tuple $(Q, A, B, \delta, q_0, q_a)$, where Q is a finite set of control states, A a finite alphabet, $B \in A$ the blank symbol, $q_0, q_a \in Q$ the initial and accepting states, and δ the transition amplitude function

$$\delta : Q \times A \times Q \times A \times \{L, R\} \rightarrow \mathbb{C}.$$

The transition amplitude function satisfies

$$\sum_{(q_2, a_2, d) \in Q \times A \times \{L, R\}} |\delta(q_1, a_1, q_2, a_2, d)|^2 = 1,$$

for any $(q_1, a_1) \in Q \times A$. L and R stand for moving left and right, respectively.

A **quantum bit** (qubit) is a two-level quantum system, modeled by the two-dimensional Hilbert space H_2 , with basis $\{|0\rangle, |1\rangle\}$. For example, for a spin- $\frac{1}{2}$ particle, $|0\rangle$ and $|1\rangle$ denote the spin-down and spin-up states, respectively. They can be mapped to the standard basis for H_2 as $|0\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix}^T$ and $|1\rangle = \begin{bmatrix} 0 & 1 \end{bmatrix}^T$.

A **quantum register** of length n is an ordered system of n qubits, modeled by the 2^n -dimensional Hilbert space $H_{2^n} = H_2 \otimes H_2 \otimes \dots \otimes H_2$ with basis $\{|00\dots 00\rangle, |00\dots 01\rangle, |00\dots 10\rangle, |00\dots 11\rangle, \dots, |11\dots 11\rangle\}$. The basis states are ordered in lexicographic order. We may also write each basis state as $|i\rangle$ for $0 \leq i < 2^n$, interpreting the n -bit string of 0s and 1s as the binary representation of i .

An **one-bit quantum gate** is a unitary map $U : H_2 \rightarrow H_2$.

An **n -bit quantum gate** is a unitary map $U : H_2 \otimes H_2 \otimes \dots \otimes H_2 \rightarrow H_2 \otimes H_2 \otimes \dots \otimes H_2$.

A **quantum circuit** on n bits is a unitary map on H_{2^n} , which can be represented by a concatenation of a finite number of quantum gates.

Facts:

1. [BV93] Any function which is computable by a TM is computable by a QTM.
2. [Fey82] Any function that is computable by a QTM is computable by a TM.
3. [NC00, pp. 13] A general state of a qubit is a unit vector $a|0\rangle + b|1\rangle$, where $a, b \in \mathbb{C}$ and $|a|^2 + |b|^2 = 1$. a and b are the probability amplitudes of $|0\rangle$ and $|1\rangle$, respectively. A measurement of a qubit yields either $|0\rangle$ or $|1\rangle$, with probability $|a|^2$ or $|b|^2$, respectively.
4. [BB02] A general state of n -bit quantum register is a unit vector

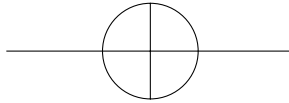
$$\sum_{x=00\dots 0}^{11\dots 1} \psi_x |x\rangle,$$

where $\psi_x \in \mathbb{C}$ and $\sum_{x=00\dots 0}^{11\dots 1} |\psi_x|^2 = 1$. ψ_x is the probability amplitude of $|x\rangle$. A measurement of a quantum register yields $|x\rangle \in \{|00\dots 0\rangle, |00\dots 1\rangle, \dots, |11\dots 1\rangle\}$, with probability $|\psi_x|^2$.

Examples:

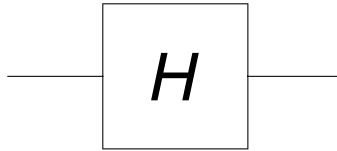
This section contains a list of quantum gates that are frequently used. We provide the description of their effects on the basis states, matrix representations, and circuit diagrams. In the circuit diagrams, the horizontal lines stand for the qubits. When there is no gate on the line, no operation is done, which can be interpreted as the identity operation. When the diagram of a gate shows up on a horizontal line/lines, the corresponding gate operation is applied to the qubit/qubits, with the input coming from the left and the output (result) going out to the right. The entire circuit diagram is read from left to right.

1. NOT gate Λ_0 : $\Lambda_0|0\rangle = |1\rangle$, $\Lambda_0|1\rangle = |0\rangle$, or $\Lambda_0 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.



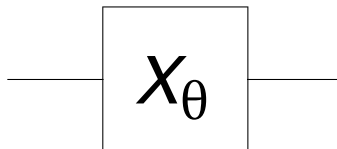
2. The Walsh–Hadamard gate H : $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, or

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$



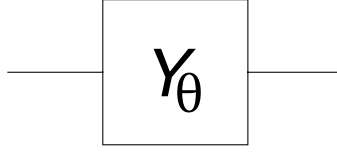
3. The x -rotation gate X_θ : $X_\theta|0\rangle = \cos \frac{\theta}{2}|0\rangle - i \sin \frac{\theta}{2}|1\rangle$, $X_\theta|1\rangle = -i \sin \frac{\theta}{2}|0\rangle + \cos \frac{\theta}{2}|1\rangle$, or

$$X_\theta = \begin{bmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}.$$



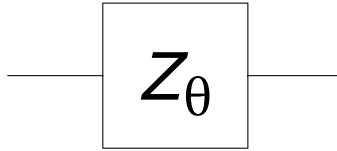
4. The y -rotation gate Y_θ : $Y_\theta|0\rangle = \cos \frac{\theta}{2}|0\rangle + \sin \frac{\theta}{2}|1\rangle$, $Y_\theta|1\rangle = -\sin \frac{\theta}{2}|0\rangle + \cos \frac{\theta}{2}|1\rangle$, or

$$Y_\theta = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}.$$



5. The z -rotation gate Z_θ : $Z_\theta|0\rangle = e^{-i\theta/2}|0\rangle$, $Z_\theta|1\rangle = e^{i\theta/2}|1\rangle$, or

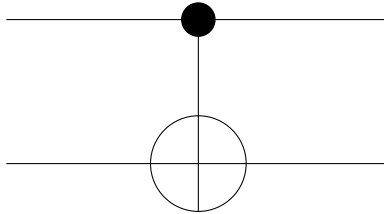
$$Z_\theta = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}.$$



6. Controlled-NOT gate Λ_1 : $\Lambda_1|00\rangle = |00\rangle$, $\Lambda_1|01\rangle = |01\rangle$, $\Lambda_1|10\rangle = |11\rangle$, $\Lambda_1|11\rangle = |10\rangle$, or,

$$\Lambda_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

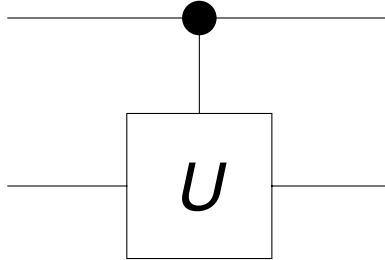
The first qubit acts as the control bit; the operation on the second qubit is controlled by it. If the control bit is $|0\rangle$, no operation is done on the second qubit. If the control bit is $|1\rangle$, the NOT gate is applied to the second qubit. There is no change on the control bit in either case. In the diagram for the Controlled-NOT gate, a black dot denotes the control bit and a vertical line signifies the control action.



7. Two-bit Controlled- U gate $\Lambda_1(U)$, where U is any arbitrary one-bit unitary transform: $\Lambda_1(U)|00\rangle = |00\rangle$, $\Lambda_1(U)|01\rangle = |01\rangle$, $\Lambda_1(U)|10\rangle = |1\rangle U|0\rangle$, $\Lambda_1(U)|11\rangle = |1\rangle U|1\rangle$, or,

$$\Lambda_1(U) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & & \\ & & U & \\ 0 & 0 & & \end{bmatrix}.$$

The first qubit acts as the control bit; the operation on the second qubit is controlled by it. If the control bit is $|0\rangle$, no operation is done on the second qubit. If the control bit is $|1\rangle$, the one-bit gate U is applied to the second qubit. There is no change on the control bit in either case. In the diagram for the Controlled- U gate, a black dot denotes the control bit and a vertical line signifies the control action.

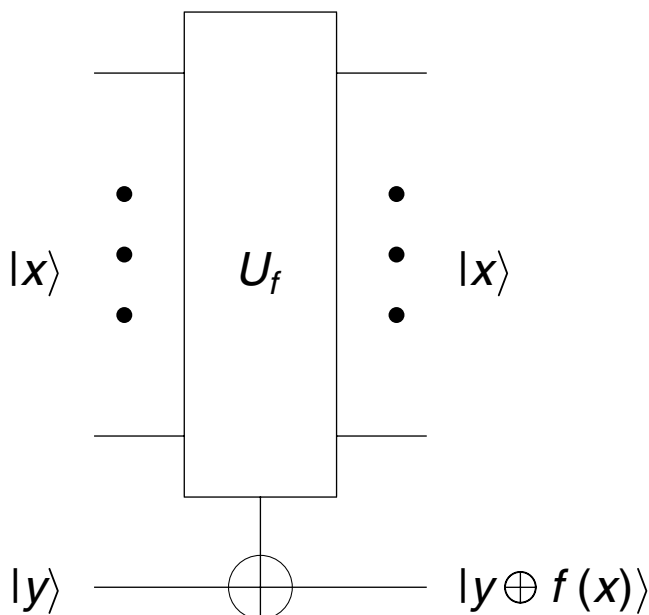


8. Function evaluation operator $U_f : H_{2^n} \otimes H_{2^m} \rightarrow H_{2^n} \otimes H_{2^m}$, where $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$, $|x\rangle \in H_{2^n}$, and $|y\rangle \in H_{2^m}$, is given by

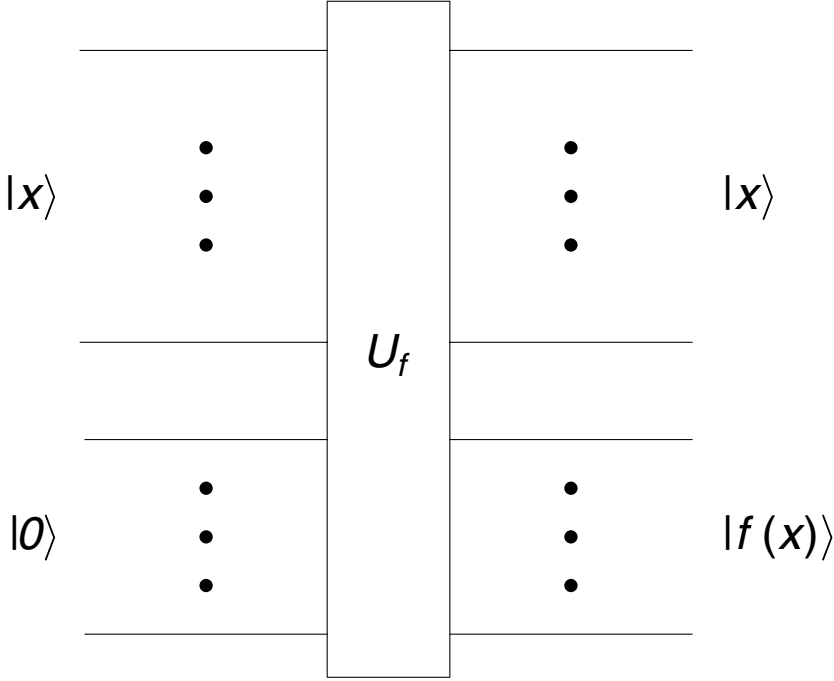
$$|x\rangle|y\rangle \rightarrow |x\rangle|y + f(x) \bmod 2^m\rangle.$$

Two special cases of this operator will be used in the algorithms discussed later.

- $m = 1$, U_f is given by: $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$, where $\oplus$ is the addition mod 2.



- $y = 0$, U_f is given by: $|x\rangle|0\rangle \rightarrow |x\rangle|f(x)\rangle$.



9. Quantum Fourier transform (QFT) (n -bit):

$$|x\rangle \rightarrow \frac{1}{2^{n-1}} \sum_{y=0}^{2^n-1} e^{2\pi i xy/2^n} |y\rangle,$$

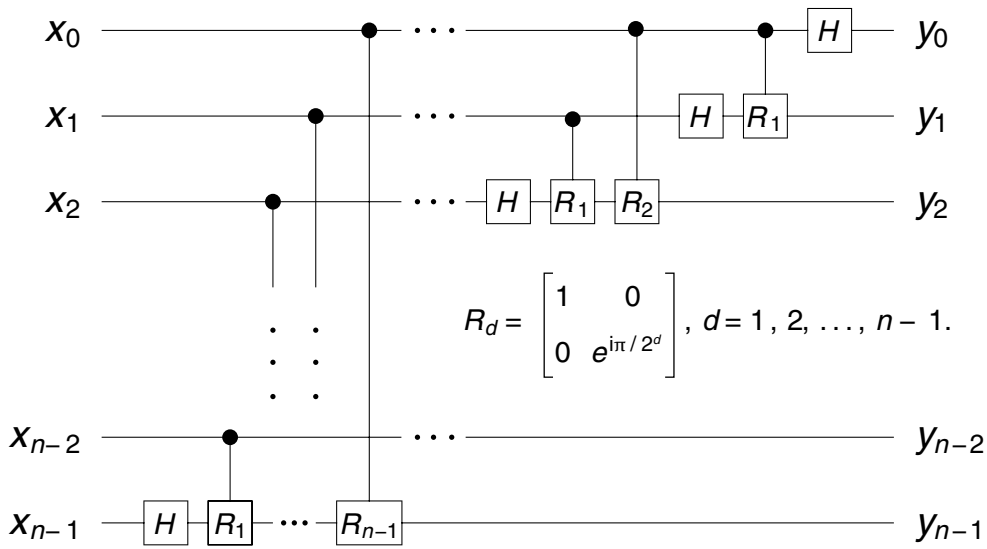
where $x \in \{0, 1, \dots, 2^n - 1\}$. This operator is a crucial building block of Shor's Factorization Algorithm. The following example manifests the power of QFT.

Example: Define a function $f : \{0, 1, 2, 3\} \rightarrow \{0, \frac{1}{\sqrt{2}}\}$ by $f(1) = f(3) = \frac{1}{\sqrt{2}}$ and $f(0) = f(2) = 0$. This function has period 2. Consider a 2-bit quantum system with state $\frac{1}{\sqrt{2}}(|1\rangle + |3\rangle)$, where the probability amplitudes of the basis states $|0\rangle$, $|1\rangle$, $|2\rangle$, and $|3\rangle$ are specified by the function values of f at 0, 1, 2, and 3. Apply QFT to this state.

$$\begin{aligned} & \frac{1}{\sqrt{2}}(|1\rangle + |3\rangle) \\ & \rightarrow \frac{1}{2\sqrt{2}}(|0\rangle + i|1\rangle - |2\rangle - i|3\rangle) + \frac{1}{2\sqrt{2}}(|0\rangle - i|1\rangle - |2\rangle + i|3\rangle) \\ & = \frac{1}{\sqrt{2}}(|0\rangle - |2\rangle). \end{aligned}$$

Measurement of the result yields 2, the period of f , with probability $\frac{1}{2}$. Thus, QFT provides a tool for period finding.

In the diagram for QFT, $x_{n-1}x_{n-2} \dots x_2x_1x_0$ and $y_{n-1}y_{n-2} \dots y_2y_1y_0$ are the binary representations of x and y , respectively.



62.2 Universal Quantum Gates

Definitions:

A 1-bit gate A is **special** if $\det(A) = 1$.

A 2-bit gate V is **primitive** if V is decomposable, i.e., there exist 1-bit gates S and T such that $V|xy\rangle = S|x\rangle \otimes T|y\rangle$, or $V|xy\rangle = S|y\rangle \otimes T|x\rangle$, for any state $|xy\rangle$.

A 2-bit gate V is **imprimitive** if it is not primitive.

A collection of quantum gates G is **universal** if, for each $n \in \mathbb{N}$, every n -bit quantum gate can be approximated with arbitrary accuracy by a circuit consisting of quantum gates in G .

A collection of quantum gates G is **exactly universal** if, for each $n \in \mathbb{N}$, every n -bit quantum gate can be obtained exactly by a circuit consisting of quantum gates in G .

Facts:

The following facts can be found in [BB02].

1. The collection of all 1-bit gates and any imprimitive 2-bit gate is universal.
2. The collection of all 1-bit gates and any imprimitive 2-bit gate is exactly universal.
3. The collection of all special 1-bit gates and any imprimitive 2-bit gate V with $\det(V)$ not being a root of unity is universal.

Examples:

1. The X_θ , Y_θ , and Z_θ gates are special.
2. The NOT gate and Walsh–Hadamard gate are not special.
3. The 2-bit gate $V = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$ is primitive, $V|xy\rangle = H|x\rangle \otimes |y\rangle$.

4. The Controlled-NOT gate is imprimitive.
5. The collection of all 1-bit gates and Controlled-NOT gate is universal, and exactly universal.
6. The collection of all X_θ , Y_θ , and Z_θ gates and Controlled-phase gate $\Lambda_1(Q_\theta)$, where $Q_\theta = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}$ and $e^{i\theta}$ is not a root of unity, is universal.

62.3 Deutsch's Problem

Definitions:

A **Boolean function** is a function with codomain $\{0, 1\}$.

A Boolean function $f : \{0, 1\} \rightarrow \{0, 1\}$ is **constant** if $f(0) = f(1)$.

A Boolean function $f : \{0, 1\} \rightarrow \{0, 1\}$ is **balanced** if $f(0) \neq f(1)$.

Problem: Deutsch

Given a Boolean function $f : \{0, 1\} \rightarrow \{0, 1\}$, determine whether it is constant or balanced.

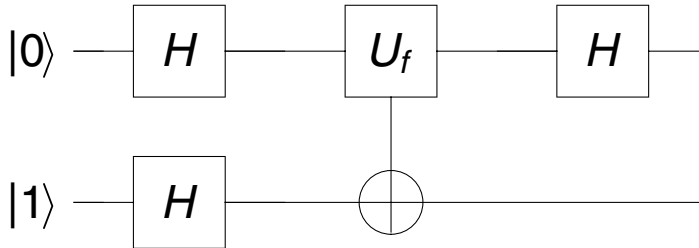
Algorithm: Deutsch

1. Prepare a two-bit quantum register and initialize it to the state $|0\rangle|1\rangle$.
2. Apply the Walsh–Hadamard transform to both qubits.
3. Apply the function evaluation operator U_f

$$U_f : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle.$$

4. Apply the Walsh–Hadamard transform to the first qubit.
5. Measure the first qubit. If the outcome is $|0\rangle$, f is constant. If the outcome is $|1\rangle$, f is balanced.

Circuit Diagram: Deutsch



Facts:

The following facts can be found in [CEM98].

1. With the classical computer, we need two evaluations of f to determine whether f is constant or balanced.
2. With the quantum computer, we need only one evaluation of f to determine whether f is constant or balanced.

Examples:

1. Let $f(0) = 0$ and $f(1) = 1$. The following sequence of quantum states shows the result of computation utilizing Deutsch's Algorithm. We start from the initial state $|0\rangle|1\rangle$. Then,

$$\begin{aligned}
& |0\rangle|1\rangle \\
& \rightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& \rightarrow \frac{|0\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|1\rangle}{\sqrt{2}} \frac{|1\rangle - |0\rangle}{\sqrt{2}} \\
& = \frac{|0\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& \rightarrow |1\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first qubit is $|1\rangle$, hence, f is balanced.

2. Let $f(0) = 0$ and $f(1) = 0$. The following sequence of quantum states shows the result of computation utilizing Deutsch's Algorithm. We start from the initial state $|0\rangle|1\rangle$. Then,

$$\begin{aligned}
& |0\rangle|1\rangle \\
& \rightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& \rightarrow \frac{|0\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& \rightarrow |0\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first qubit is $|0\rangle$, hence, f is constant.

62.4 Deutsch–Jozsa Problem

Definitions:

A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is **constant** if $f(x) = c$, $c \in \{0, 1\}$, for all $x \in \{0, 1\}^n$.

A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is **balanced** if the function value is 1 (or 0) for exactly half of the input values, i.e., $\text{card}(f^{-1}(\{1\})) = \text{card}(f^{-1}(\{0\}))$.

Problem: Deutsch–Jozsa

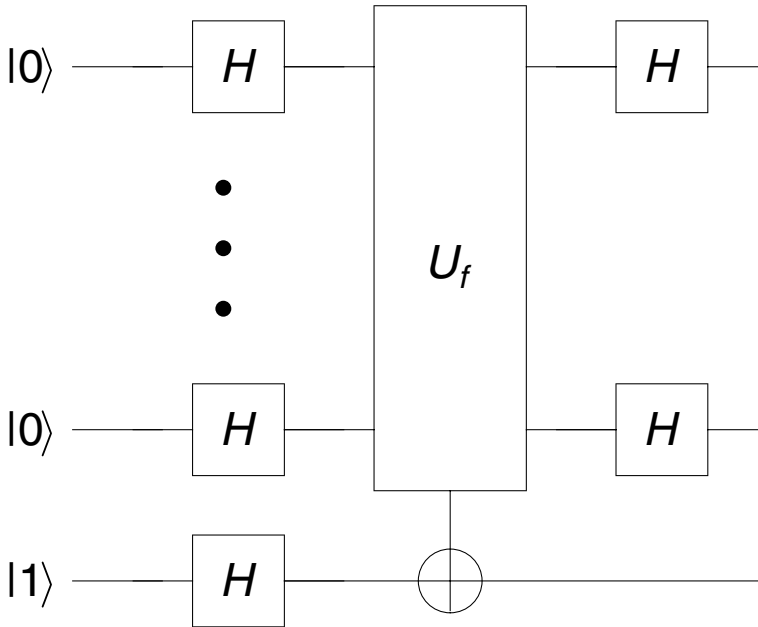
Given a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, which is either constant or balanced, determine whether it is constant or balanced.

Algorithm: Deutsch–Jozsa

1. Prepare an $(n + 1)$ -bit quantum register and initialize it to the state $(|0\rangle)^n|1\rangle$.
2. Apply the Walsh–Hadamard transform to all the qubits.
3. Apply the function evaluation operator U_f :

$$U_f : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle.$$

4. Apply the Walsh–Hadamard transform to the first n qubits.
5. Measure the first n qubit. If the outcome is $|00 \dots 0\rangle$, f is constant. If the outcome is not $|00 \dots 0\rangle$, f is balanced.

Circuit Diagram: Deutsch–Jozsa**Facts:**

The following facts can be found in [CEM98].

1. With the classical computer, we need at least $2^{n-1} + 1$ evaluations of f to determine with certainty whether f is constant or balanced.
2. With the quantum computer, we need only one evaluation of f to determine with certainty whether f is constant or balanced.

Examples:

1. Let $n = 2$ and $f(00) = f(01) = f(10) = f(11) = 1$. The following sequence of quantum states shows the result of computation utilizing Deutsch–Jozsa’s Algorithm. We start from the initial state $|00\rangle|1\rangle$. Then,

$$\begin{aligned}
& |00\rangle|1\rangle \\
\rightarrow & \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
= & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
\rightarrow & \frac{|00\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} \\
= & -\frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
= & \frac{-|00\rangle - |01\rangle - |10\rangle - |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = -\frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
\rightarrow & -|00\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first 2 qubit is $|00\rangle$, hence, f is constant.

2. Let $n = 2$, $f(00) = f(01) = 0$, and $f(10) = f(11) = 1$. The following sequence of quantum states shows the result of computation utilizing Deutsch–Jozsa’s Algorithm. We start from the initial state $|00\rangle|1\rangle$. Then,

$$\begin{aligned}
& |00\rangle|1\rangle \\
\rightarrow & \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
= & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
\rightarrow & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} \\
= & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
= & \frac{|00\rangle + |01\rangle - |10\rangle - |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
\rightarrow & |10\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first 2 qubit is $|10\rangle$, hence, f is balanced.

62.5 Bernstein–Vazirani Problem

Definitions:

Let $x = x_1x_2 \dots x_n$ and $y = y_1y_2 \dots y_n$ be two n -bit strings from $\{0, 1\}^n$. The **dot product** $x \cdot y$ is the mod 2 sum of the bitwise products:

$$x \cdot y = x_1y_1 \oplus x_2y_2 \oplus \dots \oplus x_ny_n.$$

Problem: Bernstein–Vazirani

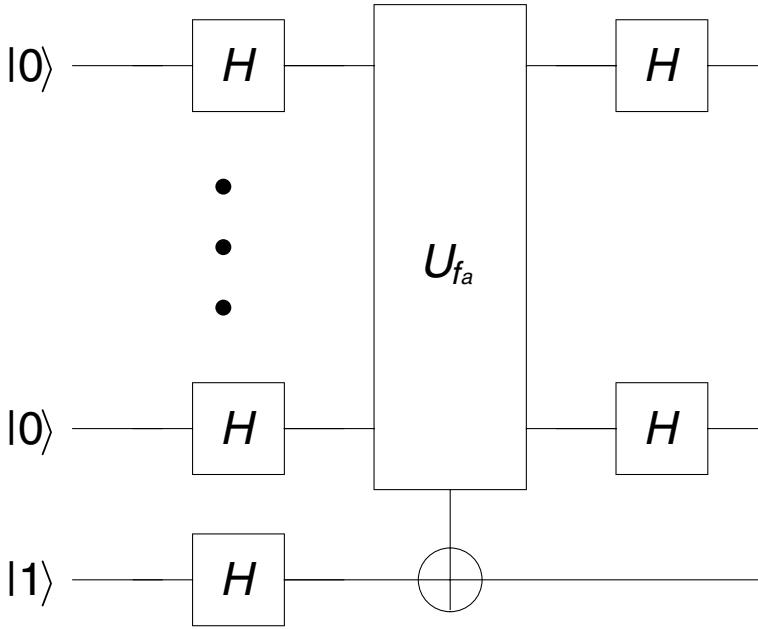
Given a Boolean function $f_a : \{0, 1\}^n \rightarrow \{0, 1\}$ defined by $f_a(x) = a \cdot x$, where a is an unknown n -bit string in $\{0, 1\}^n$, determine the value of a .

Algorithm: Bernstein–Vazirani

1. Prepare an $(n + 1)$ -bit quantum register and initialize it to the state $(|0\rangle)^n|1\rangle$.
2. Apply the Walsh–Hadamard transform to all the qubits.
3. Apply the function evaluation operator U_{f_a} :

$$U_{f_a} : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f_a(x)\rangle.$$

4. Apply the Walsh–Hadamard transform to the first n qubits.
5. Measure the first n qubits. The outcome is $|a\rangle$.

Circuit Diagram: Bernstein–Vazirani**Facts:**

The following facts can be found in [BV93].

1. With the classical computer, we need n evaluations of f_a to determine the value of a .
2. With the quantum computer, we need only one evaluation of f_a to determine the value of a .

Examples:

Let $a = 11$, a 2-bit string. The following sequence of quantum states shows the result of computation utilizing Bernstein–Vazirani’s Algorithm. We start from the initial state $|00\rangle|1\rangle$. Then,

$$\begin{aligned}
 & |00\rangle|1\rangle \\
 \rightarrow & \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
 = & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}}
 \end{aligned}$$

$$\begin{aligned}
& \rightarrow \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& = \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} - \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& = \frac{|00\rangle - |01\rangle - |10\rangle + |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
& \rightarrow |11\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first 2 qubits is $|11\rangle$, hence, $a = 11$.

62.6 Simon's Problem

Definitions:

A function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is **2-1** if for each $z \in \text{range}(f)$, there are exactly two distinct n -bit strings x and y such that $f(x) = f(y) = z$.

A function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ has a **period** a if $f(x) = f(x \oplus a)$, $\forall x \in \{0, 1\}^n$.

Problem: Simon

Given a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ which is 2-1 and has period a , determine the period a .

Algorithm: Simon

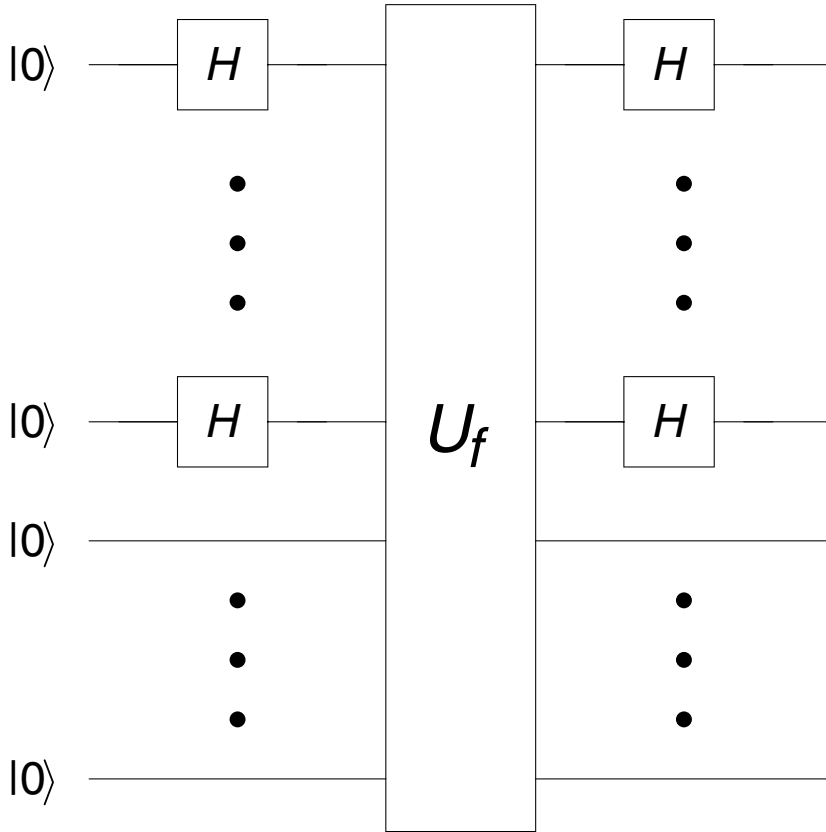
1. Repeat the following procedure for n times.
 - (a) Prepare a $2n$ -bit quantum register and initialize it to the state $(|0\rangle)^n(|0\rangle)^n$.
 - (b) Apply the Walsh–Hadamard transform to the first n qubits.
 - (c) Apply the function evaluation operator U_f :

$$U_f : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle.$$

- (d) Apply the Walsh–Hadamard transform to the first n qubits.
 - (e) Measure the first n qubits. Record the outcome $|x\rangle$.
2. With the n outcomes $x_1, x_2, \dots, x_n$, solve the following system of linear equations:

$$\begin{cases} x_1 \cdot a = 0 \\ x_2 \cdot a = 0 \\ \vdots \\ x_n \cdot a = 0. \end{cases}$$

The solution a is the period of f .

Circuit Diagram: Simon**Facts:**

The following facts can be found in [Sim94].

1. With the classical computer, we need exponentially many evaluations of f to determine the period a .
2. With the quantum computer, we need $O(n)$ evaluations (on average) of f to determine the period a .

Examples:

Let $f(00) = 01$, $f(01) = 11$, $f(10) = 01$, and $f(11) = 11$. The following sequence of quantum states shows the result of computation utilizing Simon's Algorithm. We start from the initial state $|00\rangle|00\rangle$. Then,

$$\begin{aligned}
 & |00\rangle|00\rangle \\
 \rightarrow & \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} |00\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} |00\rangle \\
 = & \frac{|00\rangle|00\rangle}{2} + \frac{|01\rangle|00\rangle}{2} + \frac{|10\rangle|00\rangle}{2} + \frac{|11\rangle|00\rangle}{2} \\
 \rightarrow & \frac{|00\rangle|01\rangle}{2} + \frac{|01\rangle|11\rangle}{2} + \frac{|10\rangle|01\rangle}{2} + \frac{|11\rangle|11\rangle}{2}
 \end{aligned}$$

$$\begin{aligned}
&= \frac{|00\rangle + |10\rangle}{2}|01\rangle + \frac{|01\rangle + |11\rangle}{2}|11\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle}{\sqrt{2}}|01\rangle + \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|1\rangle}{\sqrt{2}}|11\rangle \\
&\rightarrow |0\rangle \frac{|0\rangle + |1\rangle}{2}|01\rangle + |0\rangle \frac{|0\rangle - |1\rangle}{2}|11\rangle = \frac{|00\rangle + |01\rangle}{2}|01\rangle + \frac{|00\rangle - |01\rangle}{2}|11\rangle.
\end{aligned}$$

The outcome of measuring the first 2 qubits yields either $|00\rangle$ or $|01\rangle$. Suppose that we have run the computation above twice and obtained $|00\rangle$ and $|01\rangle$, respectively. We now have a system of linear equations:

$$\begin{cases} 00 \cdot a = 0 \\ 01 \cdot a = 0. \end{cases}$$

The solution is $a = 10$, the period of this function.

62.7 Grover's Search Algorithm

Problem: Grover

Given an unsorted database with N items, find a target item w . This problem can be formulated using an oracle function $f : \{0, 1, \dots, N-1\} \rightarrow \{0, 1\}$, where

$$f(x) = \begin{cases} 0, & \text{if } x \neq w, \\ 1, & \text{if } x = w. \end{cases}$$

Given such an oracle function, find the w such that $f(w) = 1$.

Algorithm: Grover

Without loss of generality, let $N = 2^n$.

1. Prepare an $(n+1)$ -bit quantum register and initialize it to the state $(|0\rangle)^n|1\rangle$.
2. Apply the Walsh–Hadamard transform to all the $n+1$ qubits.
3. Repeat the following procedure for about $\frac{\pi}{4}\sqrt{N}$ times. Cf. Figure 62.1.

(a) Apply the function evaluation operator U_f (selective sign flipping operator):

$$U_f : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle.$$

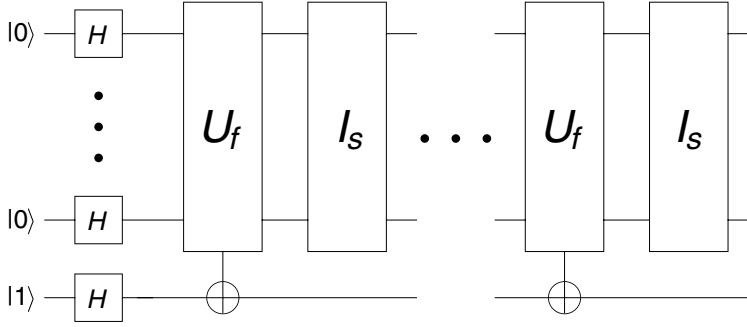
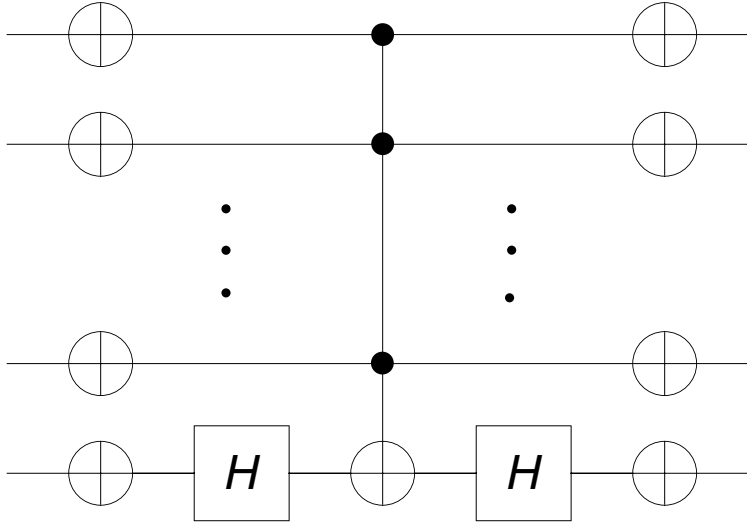
This is equivalent to the unitary operator $\mathcal{I}_w = I - 2|w\rangle\langle w|$ on the first n qubits.

(b) Apply unitary operator (inversion about the average operator) $\mathcal{I}_s = 2|s\rangle\langle s| - I$ on the first n qubits. Cf. Figure 62.2.

4. Measure the first n qubits. We obtain the search target with high probability.

Facts:

1. [Gro97] With the classical computer, on average, we need $O(N)$ oracle calls (evaluations of f) to find the search target.
2. [Gro97] With the quantum computer, on average, we need $O(\sqrt{N})$ oracle calls to find the search target.

Circuit Diagrams: Grover**FIGURE 62.1** Grover's Algorithm.**FIGURE 62.2** Inversion about the average operator $\mathcal{I}_s$.

3. [BBH98] When $N = 4$, using Grover's Algorithm, exactly one oracle call suffices to find the search target with certainty.

Examples:

Let $N = 2^2 = 4$ and Item 3 be the search target, which is encoded by the quantum state $|11\rangle$ (11 is the binary representation of 3). The following sequence of quantum states shows the result of computation utilizing Grover's Algorithm. We start from the initial state $|00\rangle|1\rangle$. Then,

$$\begin{aligned}
 & |00\rangle|1\rangle \\
 \rightarrow & \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
 = & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
 \rightarrow & \frac{|00\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|01\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|10\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} + \frac{|11\rangle}{2} \frac{|1\rangle - |0\rangle}{\sqrt{2}}
 \end{aligned}$$

$$\begin{aligned}
&= \frac{|00\rangle |0\rangle - |1\rangle}{2\sqrt{2}} + \frac{|01\rangle |0\rangle - |1\rangle}{2\sqrt{2}} + \frac{|10\rangle |0\rangle - |1\rangle}{2\sqrt{2}} - \frac{|11\rangle |0\rangle - |1\rangle}{2\sqrt{2}} \\
&= \frac{|00\rangle + |01\rangle + |10\rangle - |11\rangle}{2} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
&\rightarrow |11\rangle \frac{|0\rangle - |1\rangle}{\sqrt{2}}.
\end{aligned}$$

The outcome of measuring the first 2 qubits yields $|11\rangle$, which is the search target $w = 3$.

Comments:

Grover's Algorithm was discovered by L.K. Grover of Bell Labs in 1996. This algorithm provides a quadratic speedup over classical algorithms. Although it is not exponentially fast (as Shor's Algorithm is), it has a wide range of applications. It could be used to accelerate any algorithms related to searching an unsorted database, including quantum database search, finding the solution of NP problems, finding the median and minimum of a data set, and breaking the Data Encryption Standard (DES) cryptography system.

62.8 Shor's Factorization Algorithm

Integer Factorization Problem:

Given a composite positive integer N , factor it into the product of its prime factors.

Algorithm: Shor

1. Choose a random number $a < N$; make sure that a and N are coprime. This can be done by using a random number generator and Euclidean algorithm on a classical computer.
2. Find the period T of function $f_{a,N}(x) = a^x \bmod N$. This step can be further expanded as follows:

- (a) Prepare two L -bit quantum registers in initial state

$$\left(\frac{1}{\sqrt{2^L}} \sum_{x=0}^{2^L-1} |x\rangle \right) |0\rangle,$$

where L is chosen such that $N^2 \leq 2^L < 2N^2$.

- (b) Apply the function evaluation operator $U_f : |x\rangle|0\rangle \rightarrow |x\rangle|f_{a,N}(x)\rangle$:

$$\frac{1}{\sqrt{2^L}} \sum_{x=0}^{2^L-1} |x\rangle|0\rangle \rightarrow \frac{1}{\sqrt{2^L}} \sum_{x=0}^{2^L-1} |x\rangle|f_{a,N}(x)\rangle.$$

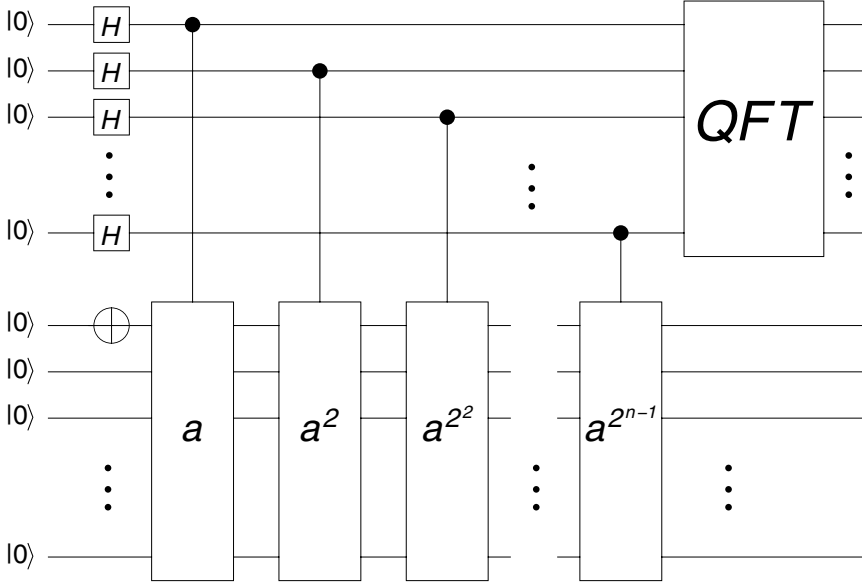
- (c) Apply QFT to the first register:

$$\frac{1}{\sqrt{2^L}} \sum_{x=0}^{2^L-1} |x\rangle|f_{a,N}(x)\rangle \rightarrow \frac{1}{2^L} \sum_{y=0}^{2^L-1} \left(\sum_{x=0}^{2^L-1} e^{2\pi i xy/2^L} |y\rangle \right) |f(x)\rangle.$$

- (d) Make a measurement on the first register, obtaining y .

- (e) Find T from y via the continued fraction for $\frac{y}{2^L}$. This step might fail; in that case, repeat from 2a.

3. If T is odd, repeat from Step 1. If T is even and $N \mid (a^{T/2} + 1)$, repeat from Step 1. If T is even and $N \nmid (a^{T/2} + 1)$, compute $d = \gcd(a^{T/2} - 1, N)$, which is a nontrivial factor of N .

Circuit Diagrams: Shor**Facts:**

The following facts can be found in [Sho94].

1. The integer factorization problem is classically *intractable*. The most efficient classical algorithm to date, a *number field sieve*, has a time complexity of $O(\exp(\log N)^{1/3}(\log \log N)^{2/3})$.
2. Shor's quantum factorization algorithm has a time complexity of $O((\log N)^2 \log \log N \log \log \log N)$. Hence, it is a polynomial time algorithm.

Examples:

Let $N = 15$. Choose $L = 8$ such that $N^2 = 225 < 2^L < 450 = 2N^2$. Choose a random integer $a = 2$, which is coprime with 15. Thus, $f_{a,N}(x) = 2^x \bmod 15$. The following sequence of quantum states shows the result of computation utilizing Shor's Algorithm:

$$\begin{aligned}
 |0\rangle|0\rangle &\rightarrow \frac{1}{2^4} \sum_{x=0}^{2^8-1} |x\rangle|0\rangle \\
 &\rightarrow \frac{1}{2^4} \sum_{x=0}^{2^8-1} |x\rangle|f_{a,N}(x)\rangle \\
 &= \frac{1}{2^4} (|0\rangle|1\rangle + |1\rangle|2\rangle + |2\rangle|4\rangle + |3\rangle|8\rangle \\
 &\quad + |4\rangle|1\rangle + |5\rangle|2\rangle + |6\rangle|4\rangle + |7\rangle|8\rangle + \cdots + |2^8-2\rangle|4\rangle + |2^8-1\rangle|8\rangle) \\
 &\rightarrow \frac{1}{2^4} \sum_{x=0}^{2^8-1} \frac{1}{2^4} \sum_{y=0}^{2^8-1} \omega^{xy} |y\rangle|2^x \bmod 15\rangle \\
 &\rightarrow \frac{1}{2^8} \sum_{y=0}^{2^8-1} |y\rangle \sum_{x=0}^{2^8-1} \omega^{xy} |2^x \bmod 15\rangle,
 \end{aligned}$$

where $\omega = e^{2\pi i/2^8}$. Suppose that the outcome of measuring the first n qubits is $|56\rangle$. We can compute the continued fraction of $\frac{56}{256}$ to be $[0, 4, \dots]$. The second number 4 satisfies $2^4 = 1 \bmod 15$. So 4 is the period of $f_{a,N}$. $2^{4/2} - 1 = 3$ yields a factor of 15 and $15 = 3 \times 5$.

Comments:

Shor's Algorithm was discovered by P. Shor of AT&T Labs in 1994. It is the most important breakthrough in the research of quantum computation so far. It solves the integer factorization problem, an extremely hard problem for classical computers, in polynomial time. The security of the RSA cryptographic system, which is widely used nowadays over the Internet, is based on the difficulty of factoring large integers. Equipped with a quantum computer, one could easily break the RSA codes with Shor's Algorithm.

References

- [BV93] E. Bernstein and U. Vazirani. Quantum complexity theory. *Proc. of the 25th Annual ACM Symposium on the Theory of Computing*, San Diego, CA, 11–20, 1993.
- [BBH98] M. Boyer, G. Brassard, P. Hoyer, and A. Tapp. Tight bounds on quantum searching. *Fortsch. Phys.* 46:493–506, 1998.
- [BB02] J. L. Brylinski and R. Brylinski. Universal quantum gates. *Mathematics of Quantum Computation* (R. Brylinski and G. Chen, Eds.). Chapman & Hall/CRC Press, Boca Raton, FL, 101–116, 2002.
- [CEM98] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. *Proc. R. Soc. London A*, 454:339–354, 1998.
- [Fey82] R. Feynman. Simulating physics with computers. *Int. J. Theor. Phys.*, 21:467–488, 1982.
- [Gro97] L.K. Grover. Quantum mechanics helps in searching for a needle in a haystack. *Phys. Rev. Lett.*, 78:325–328, 1997.
- [NC00] M.A. Nielsen and I.L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, U.K., 2000.
- [Sho94] P. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. *Proc. of the 35th Annual IEEE Symposium on the Foundations of Computer Science*, Santa Fe, NM, 124–134, 1994.
- [Sim94] D. Simon. On the power of quantum computation. *Proc. of the 35th Annual IEEE Symposium on Foundations of Computer Science*, Santa Fe, NM, 116–123, 1994.

63

Information Retrieval and Web Search

Amy N. Langville

The College of Charleston

Carl D. Meyer

North Carolina State University

63.1	The Traditional Vector Space Method	63-1
63.2	Latent Semantic Indexing	63-3
63.3	Nonnegative Matrix Factorizations	63-5
63.4	Web Search	63-8
63.5	Google's PageRank	63-10
	References	63-14

Information retrieval is the process of searching within a document collection for information most relevant to a user's query. However, the type of document collection significantly affects the methods and algorithms used to process queries. In this chapter, we distinguish between two types of document collections: traditional and Web collections. Traditional information retrieval is search within small, controlled, nonlinked collections (e.g., a collection of medical or legal documents), whereas Web information retrieval is search within the world's largest and linked document collection. In spite of the proliferation of the Web, more traditional nonlinked collections still exist, and there is still a place for the older methods of information retrieval.

63.1 The Traditional Vector Space Method

Today most search systems that deal with traditional document collections use some form of the vector space method [SB83] developed by Gerard Salton in the early 1960s. Salton's method transforms textual data into numeric vectors and matrices and employs matrix analysis techniques to discover key features and connections in the document collection.

Definitions:

For a given collection of documents and for a dictionary of m terms, document i is represented by an $m \times 1$ **document vector** $\mathbf{d}_i$ whose j th element is the number of times term j appears in document i .

The **term-by-document matrix** is the $m \times n$ matrix

$$A = [\mathbf{d}_1 \mathbf{d}_2 \cdots \mathbf{d}_n]$$

whose columns are the document vectors.

Recall is a measure of performance that is defined to be

$$0 \leq \text{Recall} = \frac{\# \text{ relevant docs retrieved}}{\# \text{ relevant docs in collection}} \leq 1.$$

Precision is another measure of performance, defined to be

$$0 \leq \text{Precision} = \frac{\# \text{ relevant docs retrieved}}{\# \text{ docs retrieved}} \leq 1.$$

Query processing is the act of retrieving documents from the collection that are most related to a user's query, and the **query vector** $\mathbf{q}_{m \times 1}$ is the binary vector defined by

$$q_i = \begin{cases} 1 & \text{if term } i \text{ is present in the user's query,} \\ 0 & \text{otherwise.} \end{cases}$$

The **relevance** of document i to a query $\mathbf{q}$ is defined to be

$$\delta_i = \cos \theta_i = \mathbf{q}^T \mathbf{d}_i / \|\mathbf{q}\|_2 \|\mathbf{d}_i\|_2.$$

For a selected tolerance τ , the **retrieved documents** that are returned to the user are the documents for which $\delta_i > \tau$.

Facts:

1. The term-by-document matrix A is sparse and nonnegative, but otherwise unstructured.
2. [BB05] In practice, weighting schemes other than raw frequency counts are used to construct the term-by-document matrix because weighted frequencies can improve performance.
3. [BB05] Query weighting may also be implemented in practice.
4. The tolerance τ is usually tuned to the specific nature of the underlying document collection.
5. Tuning can be accomplished with the technique of relevance feedback, which uses a revised query vector such as $\tilde{\mathbf{q}} = \delta_1 \mathbf{d}_1 + \delta_3 \mathbf{d}_3 + \delta_7 \mathbf{d}_7$, where $\mathbf{d}_1$, $\mathbf{d}_3$, and $\mathbf{d}_7$ are the documents the user judges most relevant to a given query $\mathbf{q}$.
6. When the columns of A and $\mathbf{q}$ are normalized, as they usually are, the vector $\boldsymbol{\delta}^T = \mathbf{q}^T A$ provides the complete picture of how well each document in the collection matches the query.
7. The vector space model is efficient because A is usually very sparse, and $\mathbf{q}^T A$ can be executed in parallel, if necessary.
8. [BB05] Because of linguistic issues such as polysomes and synonyms, the vector space model provides only decent performance on query processing tasks.
9. The underlying basis for the vector space model is the standard basis $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m$, and the orthogonality of this basis can impose an unrealistic independence among terms.
10. The vector space model is a good starting place, but variations have been developed that provide better performance.

Examples:

1. Consider a collection of seven documents and nine terms (taken from [BB05]). Terms not in the system's index are ignored. Suppose further that only the titles of each document are used for indexing. The indexed terms and titles of documents are shown below.

Terms	Documents
T1: Bab(y,ies,y's)	D1: <i>Infant & Toddler First Aid</i>
T2: Child(ren's)	D2: <i>Babies and Children's Room (For Your Home)</i>
T3: Guide	D3: <i>Child Safety at Home</i>
T4: Health	D4: <i>Your Baby's Health and Safety: From Infant to Toddler</i>
T5: Home	D5: <i>Baby Proofing Basics</i>
T6: Infant	D6: <i>Your Guide to Easy Rust Proofing</i>
T7: Proofing	D7: <i>Beanie Babies Collector's Guide</i>
T8: Safety	
T9: Toddler	

The indexed terms are italicized in the titles. Also, the stems [BB05] of the terms for baby (and its variants) and child (and its variants) are used to save storage and improve performance. The term-by-document matrix for this document collection is

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

For a query on *baby health*, the query vector is

$$\mathbf{q} = [1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0]^T.$$

To process the user's query, the cosines

$$\delta_i = \cos \theta_i = \frac{\mathbf{q}^T \mathbf{d}_i}{\|\mathbf{q}\|_2 \|\mathbf{d}_i\|_2}$$

are computed. The documents corresponding to the largest elements of δ are most relevant to the user's query. For our example,

$$\delta \approx [0 \ 0.40824 \ 0 \ 0.63245 \ 0.5 \ 0 \ 0.5],$$

so document vector 4 is scored most relevant to the query on *baby health*. To calculate the recall and precision scores, one needs to be working with a small, well-studied document collection. In this example, documents $\mathbf{d}_4$, $\mathbf{d}_1$, and $\mathbf{d}_3$ are the three documents in the collection relevant to baby health. Consequently, with $\tau = .1$, the recall score is $1/3$ and the precision is $1/4$.

63.2 Latent Semantic Indexing

In the 1990s, an improved information retrieval system replaced the vector space model. This system is called Latent Semantic Indexing (LSI) [Dum91] and was the product of Susan Dumais, then at Bell Labs. LSI simply creates a low rank approximation A_k to the term-by-document matrix A from the vector space model.

Facts:

1. [Mey00] If the term-by-document matrix $A_{m \times n}$ has the singular value decomposition $A = U \Sigma V^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$, $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, then A_k is created by truncating this expansion after k terms, where k is a user tunable parameter.
2. The recall and precision measures are generally used in conjunction with each other to evaluate performance.
3. A is replaced by $A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ in the query process so that if $\mathbf{q}$ and the columns of A_k have been normalized, then the angle vector is computed as $\boldsymbol{\delta}^T = \mathbf{q}^T A_k$.
4. The truncated SVD approximation to A is optimal in the sense that of all rank- k matrices, the truncated SVD A_k is the closest to A , and

$$\|A - A_k\|_F = \min_{\text{rank}(B) \leq k} \|A - B\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}.$$

5. This rank- k approximation reduces the so-called linguistic noise present in the term-by-document matrix and, thus, improves information retrieval performance.
6. [Dum91], [BB05], [BR99], [Ber01], [BDJ99] LSI is known to outperform the vector space model in terms of precision and recall.
7. [BR99], [Ber01], [BB05], [BF96], [BDJ99], [BO98], [Blo99], [BR01], [Dum91], [HB00], [JL00], [JB00], [LB97], [WB98], [ZBR01], [ZMS98] LSI and the truncated singular value decomposition dominated text mining research in the 1990s.
8. A serious drawback to LSI is that while it might appear at first glance that A_k should save storage over the original matrix A , this is often not the case, even when $k \ll r$. This is because A is generally very sparse, but the singular vectors $\mathbf{u}_i$ and $\mathbf{v}_i^T$ are almost always completely dense. In many cases, A_k requires more (sometimes much more) storage than A itself requires.
9. A significant problem with LSI is the fact that while A is a nonnegative matrix, the singular vectors are mixed in sign. This loss of important structure means that the truncated singular value decomposition provides no textual or semantic interpretation. Consider a particular document vector, say, column 1 of A . The truncated singular value decomposition represents document 1 as

$$A_1 = \begin{bmatrix} \vdots \\ \mathbf{u}_1 \\ \vdots \end{bmatrix} \sigma_1 v_{11} + \begin{bmatrix} \vdots \\ \mathbf{u}_2 \\ \vdots \end{bmatrix} \sigma_2 v_{12} + \dots + \begin{bmatrix} \vdots \\ \mathbf{u}_k \\ \vdots \end{bmatrix} \sigma_k v_{1k},$$

so document 1 is a linear combination of the basis vectors $\mathbf{u}_i$ with the scalar $\sigma_i v_{1i}$ being a weight that represents the contribution of basis vector i in document 1. What we would really like to do is say that basis vector i is mostly concerned with some subset of the terms, but any such textual or semantic interpretation is difficult (or impossible) when SVD components are involved. Moreover, if there were textual or semantic interpretations, the orthogonality of the singular vectors would ensure that there is no overlap of terms in the topics in the basis vectors, which is highly unrealistic.

10. [Ber01], [ZMS98] It is usually a difficult problem to determine the most appropriate value of k for a given dataset because k must be large enough so that A_k can capture the essence of the document collection, but small enough to address storage and computational issues. Various heuristics have been developed to deal with this issue.

Examples:

1. Consider again the 9×7 term-by-document matrix used in section 63.1. The rank-4 approximation to this matrix is

$$A_4 = \begin{bmatrix} 0.020 & 1.048 & -0.034 & 0.996 & 0.975 & 0.027 & 0.975 \\ -0.154 & 0.883 & 1.067 & 0.078 & 0.027 & -0.033 & 0.027 \\ -0.012 & -0.019 & 0.013 & 0.004 & 0.509 & 0.990 & 0.509 \\ 0.395 & 0.058 & 0.020 & 0.756 & 0.091 & -0.087 & 0.091 \\ -0.154 & 0.883 & 1.067 & 0.078 & 0.027 & -0.033 & 0.027 \\ 0.723 & -0.144 & 0.068 & 1.152 & 0.004 & -0.012 & 0.004 \\ -0.012 & -0.019 & 0.013 & 0.004 & 0.509 & 0.990 & 0.509 \\ 0.443 & 0.334 & 0.810 & 0.776 & -0.074 & 0.091 & -0.074 \\ 0.723 & -0.144 & 0.068 & 1.152 & 0.004 & -0.012 & 0.004 \end{bmatrix}.$$

Notice that while A is sparse and nonnegative, A_4 is dense and mixed in sign. Of course, as k increases, A_k looks more and more like A . For a query on *baby health*, the angle vector is

$$\delta \approx [.244 \quad .466 \quad -.006 \quad .564 \quad .619 \quad -.030 \quad .619]^T.$$

Thus, the information retrieval system returns documents $\mathbf{d}_5, \mathbf{d}_7, \mathbf{d}_4, \mathbf{d}_2, \mathbf{d}_1$, in order from most to least relevant. As a result, the recall improves to $2/3$, while the precision is $2/5$. Adding another singular triplet and using the approximation matrix A_5 does not change the recall or precision measures, but does give a slightly different angle vector

$$\delta \approx [.244 \quad .466 \quad -.006 \quad .564 \quad .535 \quad -.030 \quad .535]^T,$$

which is better than the A_4 angle vector because the most relevant document, $\mathbf{d}_4$, *Your Baby's Health and Safety: From Infant to Toddler*, gets the highest score.

63.3 Nonnegative Matrix Factorizations

The lack of semantic interpretation due to the mixed signs in the singular vectors is a major obstacle in using LSI. To circumvent this problem, alternative low rank approximations that maintain the nonnegative structure of the original term-by-document matrix have been proposed [LS99], [LS00], [PT94], [PT97].

Facts:

1. If $A_{m \times n} \geq 0$ has rank r , then for a given $k < r$ the goal of a nonnegative matrix factorization (NMF) is to find the nearest rank- k approximation WH to A such that $W_{m \times k} \geq 0$ and $H_{k \times n} \geq 0$. Once determined, an NMF simply replaces the truncated singular value decomposition in any text mining task such as clustering documents, classifying documents, or processing queries on documents.
2. An NMF can be formulated as a constrained nonlinear least squares problem by first specifying k and then determining

$$\min \|A - WH\|_F^2 \quad \text{subject to} \quad W_{m \times k} \geq 0, \quad H_{k \times n} \geq 0.$$

The rank of the approximation (i.e., k) becomes the number of topics or clusters in a text mining application.

3. [LS99] The Lee and Seung algorithm to compute an NMF using MATLAB is as follows.

Algorithm 1: Lee–Seung NMF

```

W = abs(randn(m,k))           % initialize with random dense matrix
H = abs(randn(k,n))           % initialize with random dense matrix
for i = 1 : maxiter
    H = H .* (WTA)./(WTWH + 10-9) % 10-9 avoids division by 0
    W = W .* (AHT)./(WHHT + 10-9)
end

```

4. The objective function $\|A - WH\|_F^2$ in the Lee and Seung algorithm tends to tail off within 50 to 100 iterations. Faster algorithms exist, but the Lee and Seung algorithm is guaranteed to converge to a local minimum in a finite number of steps.
5. [Hoy02], [Hoy04], [SBP04] Other NMF algorithms contain a tunable sparsity parameter that produces any desired level of sparseness in W and H . The storage savings of the NMF over the truncated SVD are substantial.
6. Because $A_j \approx \sum_{i=1}^k W_i h_{ij}$, and because W and H are nonnegative, each column W_i can be viewed as a topic vector—if $w_{ij_1}, w_{ij_2}, \dots, w_{ij_p}$ are the largest entries in W_i , then terms $j_1, j_2, \dots, j_p$ dictate the topics that W_i represents. The entries h_{ij} measure the strength to which topic i appears in basis document j , and k is the number of topic vectors that one expects to see in a given set of documents.
7. The NMF has some disadvantages. Unlike the SVD, uniqueness and robust computations are missing in the NMF. There is no unique global minimum for the NMF (the defining constrained least squares problem is not convex in W and H), so algorithms can only guarantee convergence to a local minimum, and many do not even guarantee that.
8. Not only will different NMF algorithms produce different NMF factors, the same NMF algorithm, run with slightly different parameters, can produce very different NMF factors. For example, the results can be highly dependent on the initial values.

Examples:

1. When the term-by-document matrix of the MEDLINE dataset [Med03] is approximated with an NMF as described above with $k = 10$, the charts in Figure 63.1 show the highest weighted terms from four representative columns of W . For example, this makes it clear that W_1 represents heart related topics, while W_2 concerns blood issues, etc.

When document 5 (column A_5) from MEDLINE is expressed as an approximate linear combination of $W_1, W_2, \dots, W_{10}$ in order of the size of the entries of H_5 , which are

$$h_{95} = .1646 > h_{65} = .0103 > h_{75} = .0045 > \dots,$$

we have that

$$A_5 \approx .1646 W_9 + .0103 W_6 + .0045 W_7 + \dots$$

$$= .1646 \begin{bmatrix} \text{fatty} \\ \text{glucose} \\ \text{acids} \\ \text{ffa} \\ \text{insulin} \\ \vdots \end{bmatrix} + .0103 \begin{bmatrix} \text{kidney} \\ \text{marrow} \\ \text{dna} \\ \text{cells} \\ \text{neph.} \\ \vdots \end{bmatrix} + .0045 \begin{bmatrix} \text{hormone} \\ \text{growth} \\ \text{hgh} \\ \text{pituitary} \\ \text{mg} \\ \vdots \end{bmatrix} + \dots$$

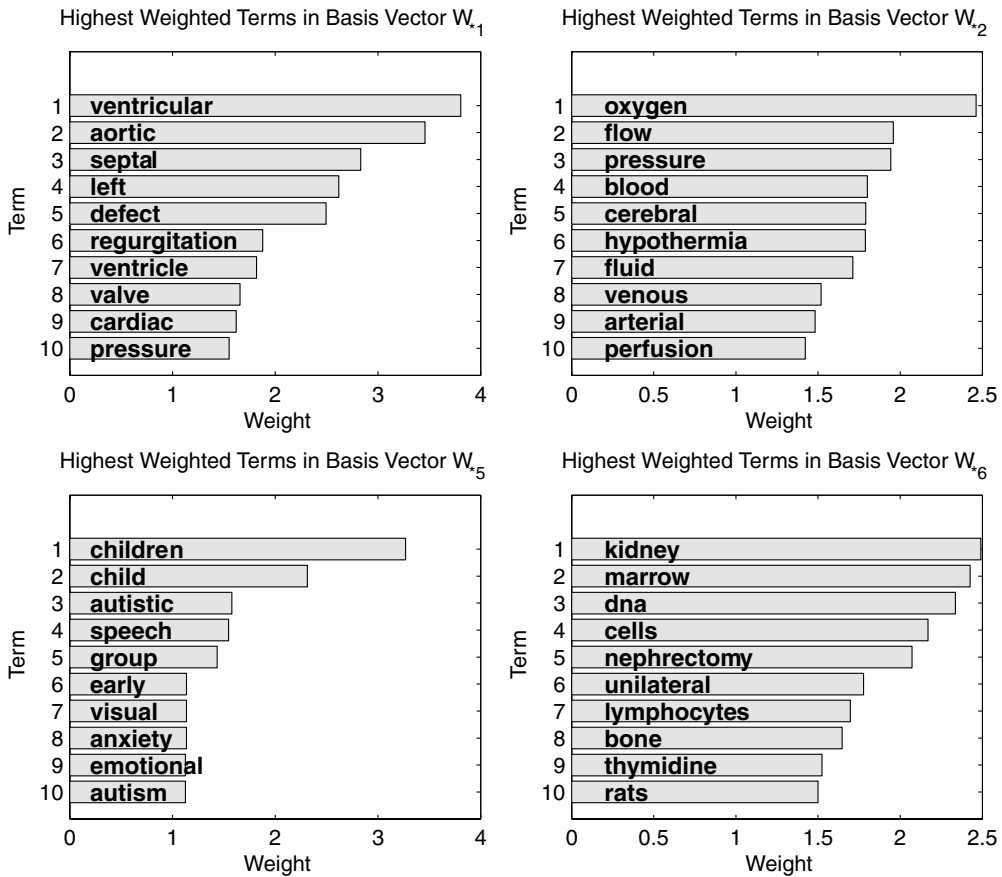


FIGURE 63.1 MEDLINE charts.

Therefore, document 5 is largely about terms contained in topic vector W_9 followed by topic vectors W_6 and W_7 .

- Consider the same 9×7 term-by-document matrix A from the example in Section 63.1. A rank-4 approximation $A_4 = W_{9 \times 4} H_{4 \times 7}$ that is produced by the Lee and Seung algorithm is

$$A_4 = \begin{bmatrix} 0.027 & 0.888 & 0.196 & 1.081 & 0.881 & 0.233 & 0.881 \\ 0 & 0.852 & 1.017 & 0.173 & 0.058 & 0.031 & 0.058 \\ 0.050 & 0.084 & 0.054 & 0.102 & 0.496 & 0.899 & 0.496 \\ 0.360 & 0.172 & 0.073 & 0.729 & 0.179 & 0.029 & 0.179 \\ 0 & 0.852 & 1.017 & 0.173 & 0.058 & 0.031 & 0.058 \\ 0.760 & 0.032 & 0.155 & 1.074 & 0.033 & 0.061 & 0.033 \\ 0.050 & 0.084 & 0.054 & 0.102 & 0.496 & 0.899 & 0.496 \\ 0.445 & 0.481 & 0.647 & 0.718 & 0.047 & 0.053 & 0.047 \\ 0.760 & 0.032 & 0.155 & 1.074 & 0.033 & 0.061 & 0.033 \end{bmatrix},$$

where

$$W_4 = \begin{bmatrix} 0.202 & 0.017 & 0.160 & 1.357 \\ 0 & 0 & 0.907 & 0.104 \\ 0.805 & 0 & 0 & 0.008 \\ 0 & 0.415 & 0 & 0.321 \\ 0 & 0 & 0.907 & 0.104 \\ 0 & 0.875 & 0 & 0.060 \\ 0.805 & 0 & 0 & 0.008 \\ 0 & 0.513 & 0.500 & 0.085 \\ 0 & 0.876 & 0 & 0.060 \end{bmatrix},$$

$$H_4 = \begin{bmatrix} 0.062 & 0.010 & 0.067 & 0.119 & 0.610 & 1.117 & 0.610 \\ 0.868 & 0 & 0.177 & 1.175 & 0 & 0.070 & 0 \\ 0 & 0.878 & 1.121 & 0.105 & 0 & 0.034 & 0 \\ 0 & 0.537 & 0 & 0.752 & 0.559 & 0 & 0.559 \end{bmatrix}.$$

Notice that both A and A_4 are nonnegative, and the sparsity of W and H makes the storage savings apparent. The error in this NMF approximation as measured by $\|A - WH\|_F^2$ is 1.56, while the error in the best rank-4 approximation from the truncated SVD is 1.42. In other words, the NMF approximation is not far from the optimal SVD approximation — this is frequently the case in practice in spite of the fact that W and H can vary with the initial conditions. For a query on *baby health*, the angle vector is

$$\delta \approx [.224 \ .472 \ .118 \ .597 \ .655 \ .143 \ .655]^T.$$

Thus, the information retrieval system that uses the nonnegative matrix factorization gives the same ranking as a system that uses the truncated singular value decomposition. However, the factors are sparse and nonnegative and can be interpreted.

63.4 Web Search

Only a few years ago, users of Web search engines were accustomed to waiting, for what would now seem to be an eternity, for search engines to return results to their queries. And when a search engine finally responded, the returned list was littered with links to information that was either irrelevant, unimportant, or downright useless. Frustration was compounded by the fact that useless links invariably appeared at or near the top of the list while useful links were deeply buried. Users had to sift through links a long way down in the list to have a chance of finding something satisfying, and being less than satisfied was not uncommon.

The reason for this is that the Web's information is not structured like information in the organized databases and document collections that generations of computer scientists had honed their techniques on. The Web is unique in the sense that it is *self-organized*. That is, unlike traditional document collections that are accumulated, edited, and categorized by trained specialists, the Web has no standards, no reviewers, and no gatekeepers to police content, structure, and format. Information on the Web is volatile and heterogeneous — links and data are rapidly created, changed, and removed, and Web information exists in multiple formats, languages, and alphabets. And there is a multitude of different purposes for Web data,

e.g., some Web pages are designed to inform while others try to sell, cheat, steal, or seduce. In addition, the Web's self organization opens the door for spammers, the nefarious people who want to illicitly commandeer your attention to sell or advertise something that you probably do not want. Web spammers are continually devising diabolical schemes to trick search engines into listing their (or their client's) Web pages near the top of the list of search results. They had an easy time of it when Web search relied on traditional IR methods based on semantic principles. Spammers could create Web pages that contained things such as miniscule or hidden text fonts, hidden text with white fonts on a white background, and misleading metatag descriptions designed to influence semantic based search engines. Finally, the enormous size of the Web, currently containing $O(10^9)$ pages, completely overwhelmed traditional IR techniques.

By 1997 it was clear that in nearly all respects the database and IR technology of the past was not well suited for Web search, so researchers set out to devise new approaches. Two big ideas emerged (almost simultaneously), and each capitalizes on the link structure of the Web to differentiate between relevant information and fluff. One approach, HITS (Hypertext Induced Topic Search), was introduced by Jon Kleinberg [Kle99], [LM06], and the other, which changed everything, is Google's *PageRank*TM that was developed by Sergey Brin and Larry Page [BP98], [BPM99], [LM06]. While variations of HITS and PageRank followed (e.g., Lempel's SALSA [LM00], [LM05], [LM06]), the basic idea of PageRank became the driving force, so the focus is on this concept.

Definitions:

Early in the game, search companies such as Yahoo![®] employed students to surf the Web and record key information about the pages they visited. This quickly overwhelmed human capability, so today all Web search engines use **Web Crawlers**, which is software that continuously scours the Web for information to return to a central repository.

Web pages found by the robots are temporarily stored in entirety in a **page repository**. Pages remain in the repository until they are sent to an indexing module, where their vital information is stripped to create a compressed version of the page. Popular pages that are repeatedly requested by users are stored here longer, perhaps indefinitely.

The **indexing module** extracts only key words, key phrases, or other vital descriptors, and it creates a compressed description of the page that can be "indexed." Depending on the popularity of a page, the uncompressed version is either deleted or returned to the page repository.

There are three general kinds of **indices** that contain compressed information for each Web page. The **content index** contains information such as key words or phrases, titles, and anchor text, and this is stored in a compressed form using an **inverted file** structure, which is simply the electronic version of a book index, i.e., each morsel of information points to a list of pages containing it. Information regarding the hyperlink structure of a page is stored in compressed form in the **structure index**. The crawler module sometimes accesses the structure index to find uncrawled pages. Finally, there are **special-purpose indices** such as an image index and a pdf index. The crawler, page repository, indexing module, and indices, along with their corresponding data files, exist and operate independent of users and their queries.

The **query module** converts a user's natural language query into a language that the search engine can understand (usually numbers), and consults the various indices in order to answer the query. For example, the query module consults the content index and its inverted file to find which pages contain the query terms.

The **pertinent pages** are the pages that contain query terms. After pertinent pages have been identified, the query module passes control to the ranking module.

The **ranking module** takes the set of pertinent pages and ranks them according to some criterion, and this criterion is the heart of the search engine — it is the distinguishing characteristic that differentiates one search engine from another. The ranking criterion must somehow discern which Web pages best respond to a user's query, a daunting task because there might be millions of pertinent pages. Unless a search engine wants to play the part of a censor (which most do not), the user is given the opportunity of seeing a list of links to a large proportion of the pertinent pages, but with less useful links permuted downward.

PageRank is Google's patented ranking system, and some of the details surrounding PageRank are discussed below.

Facts:

1. Google assigns at least two scores to each Web page. The first is a popularity score and the second is a content score. Google blends these two scores to determine the final ranking of the results that are returned in response to a user's query.
2. [BP98] The rules used to give each pertinent page a content score are trade secrets, but they generally take into account things such as whether the query terms appear in the title or deep in the body of a Web page, the number of times query terms appear in a page, the proximity of multiple query words to one another, and the appearance of query terms in a page (e.g., headings in bold font score higher). The content of neighboring Web pages is also taken into account.
3. Google is known to employ over a hundred such metrics in this regard, but the details are proprietary. While these metrics are important, they are secondary to the popularity score, which is the primary component of PageRank. The content score is used by Google only to temper the popularity score.

63.5 Google's PageRank

The popularity score is where the mathematics lies, so it is the focus of the remainder of this exposition. We will identify the term "PageRank" with just the mathematical component of Google's PageRank (the popularity score) with the understanding that PageRank may be tweaked by a content score to produce a final ranking.

Both PageRank and Google were conceived by Sergey Brin and Larry Page while they were computer science graduate students at Stanford University, and in 1998 they took a leave of absence to focus on their growing business. In a public presentation at the Seventh International World Wide Web conference (WWW98) in Brisbane, Australia, their paper "The PageRank citation ranking: Bringing order to the Web" [BPM99] made small ripples in the information science community that quickly turned into waves.

The original idea was that a *page is important if it is pointed to by other important pages*. That is, the importance of your page (its PageRank) is determined by summing the PageRanks of all pages that point to yours. Brin and Page also reasoned that when an important page points to several places, its weight (PageRank) should be distributed proportionately.

In other words, if YAHOO! points to 99 pages in addition to yours, then you should only get credit for $1/100$ of YAHOO!'s PageRank. This is the intuitive motivation behind Google's PageRank concept, but significant modifications are required to turn this basic idea into something that works in practice.

For readers who want to know more, the book *Google's PageRank and Beyond: The Science of Search Engine Rankings* [LM06] (Princeton University Press, 2006) contains over 250 pages devoted to link analysis algorithms, along with other ranking schemes such as HITS and SALSA as well as additional background material, examples, code, and chapters dealing with more advanced issues in Web search ranking.

Definitions:

The **hyperlink matrix** is the matrix $H_{n \times n}$ that represents the link structure of the Web, and its entries are given by

$$h_{ij} = \begin{cases} 1/|O_i| & \text{if there is a link from page } i \text{ to page } j, \\ 0 & \text{otherwise,} \end{cases}$$

where $|O_i|$ is the number of outlinks from page i .

Suppose that there are n Web pages, and let $r_i(0)$ denote the initial rank of the i th page. If the ranks are successively modified by setting

$$r_i(k+1) = \sum_{j \in I_i} \frac{r_j(k)}{|O_j|}, \quad k = 1, 2, 3, \dots,$$

where $r_i(k)$ is the rank of page i at iteration k and I_i is the set of pages pointing (linking) to page i , then the rankings after the k th iteration are

$$\mathbf{r}^T(k) = (r_1(k), r_2(k), \dots, r_n(k)) = \mathbf{r}^T(0)H^k.$$

The **conceptual PageRank** of the i th Web page is defined to be

$$r_i = \lim_{k \rightarrow \infty} r_i(k),$$

provided that the limit exists. However, this definition is strictly an intuitive concept because the natural structure of the Web generally prohibits these limits from existing.

A **dangling node** is a Web page that contain no out links. Dangling nodes produce zero rows in the hyperlink matrix H , so even if $\lim_{k \rightarrow \infty} H^k$ exists, the limiting vector $\mathbf{r}^T = \lim_{k \rightarrow \infty} \mathbf{r}^T(k)$ would be dependent on the initial vector $\mathbf{r}^T(0)$, which is not good.

The **stochastic hyperlink matrix** is produced by perturbing the hyperlink matrix to be stochastic. In particular,

$$S = H + \mathbf{a}\mathbf{1}^T/n, \quad (63.1)$$

where $\mathbf{a}$ is the column in which

$$a_i = \begin{cases} 1 & \text{if page } i \text{ is a dangling node,} \\ 0 & \text{otherwise.} \end{cases}$$

S is a stochastic matrix that is identical to H except that zero rows in H are replaced by $\mathbf{1}^T/n$ ($\mathbf{1}$ is a vector of 1s and $n = O(10^9)$, so entries in $\mathbf{1}^T/n$ are pretty small). The effect is to eliminate dangling nodes. Any probability vector $\mathbf{p}^T > 0$ can be used in place of the uniform vector.

The **Google matrix** is defined to be the stochastic matrix

$$G = \alpha S + (1 - \alpha)E, \quad (63.2)$$

where $E = \mathbf{1}\mathbf{v}^T$ in which $\mathbf{v}^T > 0$ can be any probability vector. Google originally set $\alpha = .85$ and $\mathbf{v}^T = \mathbf{1}^T/n$. The choice of α is discussed later in Facts 12, 13, and 14.

The **personalization vector** is the vector $\mathbf{v}^T$ in $E = \mathbf{1}\mathbf{v}^T$ in Equation 63.2. Manipulating $\mathbf{v}^T$ gives Google the flexibility to make adjustments to PageRanks as well as to personalize them (thus, the name “personalization vector”) [HKJ03], [LM04a].

The **PageRank vector** is the left Perron vector (i.e., stationary distribution) $\boldsymbol{\pi}^T$ of the Google matrix G . In particular, $\boldsymbol{\pi}^T(I - G) = 0$, where $\boldsymbol{\pi}^T > 0$ and $\|\boldsymbol{\pi}^T\|_1 = 1$. The components of this vector constitute Google’s popularity score of each Web page.

Facts:

1. [Mey00, Chap. 8] The Google matrix G is a primitive stochastic matrix, so the spectral radius $\rho(G) = 1$ is a simple eigenvalue, and 1 is the only eigenvalue on the unit circle.
2. The iteration defined by $\boldsymbol{\pi}^T(k+1) = \boldsymbol{\pi}^T(k)G$ converges, independent of the starting vector, to a unique stationary probability distribution $\boldsymbol{\pi}^T$, which is the PageRank vector.
3. The irreducible aperiodic Markov chain defined by $\boldsymbol{\pi}^T(k+1) = \boldsymbol{\pi}^T(k)G$ is a constrained random walk on the Web graph. The random walker can be characterized as a Web surfer who, at each step, randomly chooses a link from his current page to click on except that

- (a) When a dangling node is encountered, the excursion is continued by jumping to another page selected at random (i.e., with probability $1/n$).
- (b) At each step, the random Web surfer has a chance (with probability $1 - \alpha$) of becoming bored with following links from the current page, in which case the random Web surfer continues the excursion by jumping to page j with probability v_j .
4. The random walk defined by $\mathbf{r}^T(k+1) = \mathbf{r}^T(k)S$ will generally not converge because Web's graph structure is not strongly connected, which results in a reducible chain with many isolated ergodic classes.
5. The power method has been Google's computational method of choice for computing the PageRank vector. If formed explicitly, G is completely dense, and the size of n would make each power iteration extremely costly — billions of flops for each step. But writing the power method as

$$\boldsymbol{\pi}^T(k+1) = \boldsymbol{\pi}^T(k)G = \alpha \boldsymbol{\pi}^T(k)H + (\alpha \boldsymbol{\pi}^T(k)\mathbf{a})\mathbf{1}^T/n + (1 - \alpha)\mathbf{v}^T$$

shows that only extremely sparse vector-matrix multiplications are required. Only the nonzero h_{ij} 's are needed, so G and S are neither formed nor stored.

6. When implemented as shown above, each power step requires only $\text{nnz}(H)$ flops, where $\text{nnz}(H)$ is the number of nonzeros in H , and, since the average number of nonzeros per column in H is significantly less than 10, we have $O(\text{nnz}(H)) \approx O(n)$. Furthermore, the inherent parallelism is easily exploited, and the current iterate is the only vector stored at each step.
7. Because the Web has many disconnected components, the hyperlink matrix is highly reducible, and compensating for the dangling nodes to construct the stochastic matrix S does not significantly affect this.
8. [Mey00, p. 695–696] Since S is also reducible, S can be symmetrically permuted to have the form

$$S \sim \left[\begin{array}{cccc|cccc} S_{11} & S_{12} & \cdots & S_{1r} & S_{1,r+1} & S_{1,r+2} & \cdots & S_{1m} \\ 0 & S_{22} & \cdots & S_{2r} & S_{2,r+1} & S_{2,r+2} & \cdots & S_{2m} \\ \vdots & & \ddots & \vdots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & S_{rr} & S_{r,r+1} & S_{r,r+2} & \cdots & S_{rm} \\ \hline 0 & 0 & \cdots & 0 & S_{r+1,r+1} & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & S_{r+2,r+2} & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & S_{mm} \end{array} \right],$$

where the following are true.

- For each $1 \leq i \leq r$, S_{ii} is either irreducible or $[0]_{1 \times 1}$.
 - For each $1 \leq i \leq r$, there exists some $j > i$ such that $S_{ij} \neq 0$.
 - $\rho(S_{ii}) < 1$ for each $1 \leq i \leq r$.
 - $S_{r+1,r+1}, S_{r+2,r+2}, \dots, S_{m,m}$ are each stochastic and irreducible.
 - 1 is an eigenvalue for S that is repeated exactly $m - r$ times.
9. The natural structure of the Web forces the algebraic multiplicity of the eigenvalue 1 to be large.
 10. [LM04a][LM06][Mey00, Ex. 7.1.17, p. 502] If the eigenvalues of $S_{n \times n}$ are

$$\lambda(S) = \{ \underbrace{1, 1, \dots, 1}_{m-r}, \mu_{m-r+1}, \dots, \mu_n \}, \quad 1 > |\mu_{m-r+1}| \geq \dots \geq |\mu_n|,$$

then the eigenvalues of the Google matrix $G = \alpha S + (1 - \alpha)E$ are

$$\lambda(G) = \{1, \underbrace{\alpha, \dots, \alpha}_{m-r-1}, (\alpha\mu_{m-r+1}), \dots, (\alpha\mu_n)\}. \quad (63.3)$$

11. [Mey00] The asymptotic rate of convergence of any aperiodic Markov chain is governed by the magnitude of its largest subdominant eigenvalue. In particular, if the distinct eigenvalues λ_i of an aperiodic chain are ordered $\lambda_1 = 1 > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$, then the number of incorrect digits in each component of $\pi^T(k)$ is eventually going to be reduced by about $-\log_{10}|\lambda_2|$ digits per iteration.
12. Determining (or even estimating) $|\lambda_2|$ normally requires substantial effort, but Equation 63.3 says that $\lambda_2 = \alpha$ for the Google matrix G . This is an extremely happy accident because it means that Google's engineers can completely control the rate of convergence, regardless of the value of the personalization vector $\mathbf{v}^T$ in $E = \mathbf{1}\mathbf{v}^T$.
13. At the last public disclosure Google was setting $\alpha = .85$, so, at this value, the asymptotic rate of convergence of the power method is $-\log_{10}(.85) \approx .07$, which means that the power method will eventually take about 14 iterations for each significant place of accuracy that is required.
14. [LM04a] Even though they can control the rate of convergence with α , Google's engineers are forced to perform a delicate balancing act because while decreasing α increases the rate of convergence, decreasing α also lessens the effect of the true hyperlink structure of the Web, which is Google's primary mechanism for measuring Webpage importance. Increasing α more accurately reflects the Web's true link structure, but, along with slower convergence, sensitivity issues begin to surface in the sense that slightly different values for α near 1 can produce significantly different PageRanks.
15. [KHM03a] The power method can be substantially accelerated by a quadratic extrapolation technique similar to Aitken's Δ^2 method, and there is reason to believe that Google has adopted this procedure.
16. [KHM03b], [KHG04] Other improvements to the basic power method include a block algorithm and an adaptive algorithm that monitors the convergence of individual elements of the PageRank vector. As soon as components of the vector have converged to an acceptable tolerance, they are no longer computed. Convergence is faster because the work per iteration decreases as the process evolves.
17. [LM02], [LM04a], [LM04b], [LGZ03] Other methods partition H into two groups according to dangling nodes and nondangling nodes. The problem is then aggregated by lumping all of the dangling nodes into one super state to produce a problem that is significantly reduced in size — this is due to the fact that the dangling nodes account for about one fourth of the Web's nodes. The most exciting feature of all these algorithms is that they do not compete with one another. In fact, it is possible to combine some of these algorithms to achieve even greater performance.
18. The accuracy of PageRank computations is an important implementation issue, but we do not know the accuracy with which Google works. It seems that it must be at least high enough to differentiate between the often large list of ranked pages that Google usually returns, and since π^T is a probability vector containing $O(10^9)$ components, it is reasonable to expect that accuracy giving at least ten significant places is needed to distinguish among the elements.
19. A weakness of PageRank is “topic drift.” The PageRank vector might be mathematically accurate, but this is of little consequence if the results point the user to sites that are off-topic. PageRank is a query-independent measure that is essentially determined by a popularity contest with everyone on Web having a vote, and this tends to skew the results in the direction of importance (measured by popularity) over relevance to the query. This means that PageRank may have trouble distinguishing between pages that are authoritative in general and pages that are authoritative more specifically to the query topic. It is believed that Google engineers devote much effort to mitigate this problem, and this is where the metrics that determine the content score might have an effect.

20. In spite of topic drift, Google's decision to measure importance by means of popularity over relevance turned out to be the key to Google's success and the source of its strength. The query-dependent measures employed by Google's predecessors were major stumbling blocks in maintaining query processing speed as the Web grew. PageRank is query-independent, so at query time only a quick lookup into an inverted file storage is required to determine pertinent pages, which are then returned in order by the precomputed PageRanks. The small compromise of topic drift in favor of processing speed won the day.
21. A huge advantage of PageRank is its virtual imperviousness to spamming (artificially gaming the system). High PageRank is achieved by having many inlinks from highly ranked pages, and while it may not be so hard for a page owner to have many of his cronies link to his page, it is difficult to generate a lot of inlinks from *important* sites that have a high rank.
22. [TM03] Another strength of PageRank concerns the flexibility of the "personalization" (or "intervention") vector $\mathbf{v}^T$ that Google is free to choose when defining the perturbation term $E = \mathbf{1}\mathbf{v}^T$. The choice of $\mathbf{v}^T$ affects neither mathematical nor computational aspects, but it does alter the rankings in a predictable manner. This can be a terrific advantage if Google wants to intervene to push a site's PageRank down or up, perhaps to punish a suspected "link farmer" or to reward a favored client. Google has claimed that it does not make a practice of rewarding favored clients, but it is known that Google is extremely vigilant and sensitive concerning people who try to manipulate PageRank, and such sites are punished. However, the outside world is not privy to the extent to which either the stick or carrot is employed.
23. [FLM⁺04], [KH03], [KHG04], [KHM03a], [LM04a], [LM06], [LGZ03], [LM03], [NZJ01] In spite of the simplicity of the basic concepts, subtle issues such as personalization, practical implementations, sensitivity, and updating lurk just below the surface.
24. We have summarized only the mathematical component of Google's ranking system, but, as mentioned earlier, there are a hundred or more nonmathematical content metrics that are also considered when Google responds to a query. The results seen by a user are in fact PageRank tempered by these other metrics. While Google is secretive about most of its other metrics, it has made it clear that the other metrics are subservient to their mathematical PageRank scores.

Acknowledgment

This work was supported in part by the National Science Foundation under NSF grant CCR-0318575 (Carl D. Meyer).

References

- [BB05] Michael W. Berry and Murray Browne. *Understanding Search Engines: Mathematical Modeling and Text Retrieval*. SIAM, Philadelphia, 2nd ed., 2005.
- [BDJ99] Michael W. Berry, Zlatko Drmac, and Elizabeth R. Jessup. Matrices, vector spaces and information retrieval. *SIAM Rev.*, 41:335–362, 1999.
- [Ber01] Michael W. Berry, Ed. *Computational Information Retrieval*. SIAM, Philadelphia, 2001.
- [BF96] Michael W. Berry and R.D. Fierro. Low-rank orthogonal decompositions for information retrieval applications. *J. Num. Lin. Alg. Appl.*, 1(1):1–27, 1996.
- [Blo99] Katarina Blom. *Information Retrieval Using the Singular Value Decomposition and Krylov Subspaces*. Ph.D. thesis, University of Chalmers, Göteborg, Sweden, January 1999.
- [BO98] Michael W. Berry and Gavin W. O'Brien. Using linear algebra for intelligent information retrieval. *SIAM Rev.*, 37:573–595, 1998.
- [BP98] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual Web search engine. *Comp. Networks ISDN Syst.*, 33:107–117, 1998.

- [BPM99] Sergey Brin, Lawrence Page, R. Motwami, and Terry Winograd. The PageRank citation ranking: bringing order to the Web. Technical Report 1999-0120, Computer Science Department, Stanford University, 1999.
- [BR01] Katarina Blom and Axel Ruhe. Information retrieval using very short Krylov sequences. In *Computational Information Retrieval*, pp. 41–56, 2001.
- [BR99] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. *Modern Information Retrieval*. ACM Press, Pearson Education Limited, Essex, England, 1999.
- [Dum91] Susan T. Dumais. Improving the retrieval of information from external sources. *Behav. Res. Meth., Instru. Comp.*, 23:229–236, 1991.
- [FLM⁺04] Ayman Farahat, Thomas Lofaro, Joel C. Miller, Gregory Rae, and Lesley A. Ward. Existence and uniqueness of ranking vectors for linear link analysis. *SIAM J. Sci. Comp.*, 27(4): 1181–1201, 2006.
- [GL96] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 1996.
- [HB00] M.K. Hughey and Michael W. Berry. Improved query matching using kd-trees, a latent semantic indexing enhancement. *Inform. Retri.*, 2:287–302, 2000.
- [HKJ03] Taher H. Haveliwala, Sepandar D. Kamvar, and Glen Jeh. An analytical comparison of approaches to personalizing PageRank. Technical report, Stanford University, 2003.
- [Hoy02] Patrik O. Hoyer. Non-negative sparse coding. In *Neural Networks for Signal Processing XII (Proc. IEEE Workshop on Neural Networks for Signal Processing)*, Martigny, Switzerland, pp. 557–565, 2002.
- [Hoy04] Patrik O. Hoyer. Non-negative matrix factorization with sparseness constraints. *J. Mach. Learn. Res.*, 5:1457–1469, 2004.
- [JB00] Eric P. Jiang and Michael W. Berry. Solving total least squares problems in information retrieval. *Lin. Alg. Appl.*, 316:137–156, 2000.
- [JL00] Fan Jiang and Michael L. Littman. Approximate dimension equalization in vector-based information retrieval. In *The Seventeenth International Conference on Machine Learning*, Stanford University, pp. 423–430, 2000.
- [KH03] Sepandar D. Kamvar and Taher H. Haveliwala. The condition number of the PageRank problem. Technical report, Stanford University, 2003.
- [KHG04] Sepandar D. Kamvar, Taher H. Haveliwala, and Gene H. Golub. Adaptive methods for the computation of PageRank. *Lin. Alg. Appl.*, 386:51–65, 2004.
- [KHM03a] Sepandar D. Kamvar, Taher H. Haveliwala, Christopher D. Manning, and Gene H. Golub. Extrapolation methods for accelerating PageRank computations. In *Twelfth International World Wide Web Conference*, New York, 2003. ACM Press.
- [KHM03b] Sepandar D. Kamvar, Taher H. Haveliwala, Christopher D. Manning, and Gene H. Golub. Exploiting the block structure of the Web for computing PageRank. Technical Report 2003–17, Stanford University, 2003.
- [Kle99] Jon Kleinberg. Authoritative sources in a hyperlinked environment. *J. ACM*, 46, 1999.
- [LB97] Todd A. Letsche and Michael W. Berry. Large-scale information retrieval with LSI. *Inform. Comp. Sci.*, pp. 105–137, 1997.
- [LGZ03] Chris Pan-Chi Lee, Gene H. Golub, and Stefanos A. Zenios. A fast two-stage algorithm for computing PageRank and its extensions. Technical Report SCCM-2003-15, Scientific Computation and Computational Mathematics, Stanford University, 2003.
- [LM00] Ronny Lempel and Shlomo Moran. The stochastic approach for link-structure analysis (SALSA) and the TKC effect. In *The Ninth International World Wide Web Conference*, New York, 2000. ACM Press.
- [LM02] Amy N. Langville and Carl D. Meyer. Updating the stationary vector of an irreducible Markov chain. Technical Report crsc02-tr33, North Carolina State, Mathematics Dept., CRSC, 2002.
- [LM03] Ronny Lempel and Shlomo Moran. Rank-stability and rank-similarity of link-based web ranking algorithms in authority-connected graphs. In *Second Workshop on Algorithms and Models for the Web-Graph (WAW 2003)*, Budapest, Hungary, May 2003.

- [LM04a] Amy N. Langville and Carl D. Meyer. Deeper inside PageRank. *Inter. Math. J.*, 1(3):335–400, 2005.
- [LM04b] Amy N. Langville and Carl D. Meyer. A reordering for the PageRank problem. *SIAM J. Sci. Comp.*, 27(6):2112–2120, 2006.
- [LM05] Amy N. Langville and Carl D. Meyer. A survey of eigenvector methods of web information retrieval. *SIAM Rev.*, 47(1):135–161, 2005.
- [LM06] Amy N. Langville and Carl D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, Princeton, NJ, 2006.
- [LS99] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401:788–791, 1999.
- [LS00] Daniel D. Lee and H. Sebastian Seung. Algorithms for the non-negative matrix factorization. *Adv. Neur. Inform. Proc.*, 13:556–562, 2001.
- [Med03] Medlars test collection, 2003. <http://www.cs.utk.edu/~lsi/>.
- [Mey00] Carl D. Meyer. *Matrix Analysis and Applied Linear Algebra*. SIAM, Philadelphia, 2000.
- [NZJ01] Andrew Y. Ng, Alice X. Zheng, and Michael I. Jordan. Link analysis, eigenvectors and stability. In *The Seventh International Joint Conference on Artificial Intelligence*, Seattle, WA, 2001.
- [PT94] Pentti Paatero and U. Tapper. Positive matrix factorization: a non-negative factor model with optimal utilization of error estimates of data values. *Environmetrics*, 5:111–126, 1994.
- [PT97] Pentti Paatero and U. Tapper. Least squares formulation of robust non-negative factor analysis. *Chemomet. Intell. Lab. Sys.*, 37:23–35, 1997.
- [SB83] Gerard Salton and Chris Buckley. *Introduction to Modern Information Retrieval*. McGraw-Hill, New York, 1983.
- [SBP04] Farial Shahnaz, Michael W. Berry, V. Paul Pauca, and Robert J. Plemmons. Document clustering using nonnegative matrix factorization. *J. Inform. Proc. Man.*, 42(2):373–386, 2006.
- [TM03] Michael Totty and Mylene Mangalindan. As Google becomes web's gatekeeper, sites fight to get in. *Wall Street Journal*, CCXLI(39), 2003. February 26.
- [WB98] Dian I. Witter and Michael W. Berry. DOWDATING the latent semantic indexing model for conceptual information retrieval. *Comp. J.*, 41(1):589–601, 1998.
- [ZBR01] Xiaoyan Zhang, Michael W. Berry, and Padma Raghavan. Level search schemes for information filtering and retrieval. *Inform. Proc. Man.*, 37:313–334, 2001.
- [ZMS98] Hongyuan Zha, Osni Marques, and Horst D. Simon. A subspace-based model for information retrieval with applications in latent semantic indexing. *Lect. Notes Comp. Sci.*, 1457:29–42, 1998.

64

Signal Processing

64.1	Basic Concepts	64-1
64.2	Random Signals	64-4
64.3	Linear Prediction	64-7
64.4	Wiener Filtering	64-10
64.5	Adaptive Filtering	64-12
64.6	Spectral Estimation	64-14
64.7	Direction of Arrival Estimation	64-15
	References	64-18

Michael Stewart
Georgia State University

A signal is a function of time, depending on either a continuous, real time variable or a discrete integer time variable. Signal processing is a collection of diverse mathematical, statistical, and computational techniques for transforming, extracting information from, and modeling of signals. Common signal processing techniques include filtering a signal to remove undesired frequency components, extracting a model that describes key features of a signal, estimation of the frequency components of a signal, prediction of the future behavior of a signal from past data, and finding the direction from which a signal arrives at an array of sensors.

Many modern methods for signal processing have deep connections to linear algebra. Rather than attempting to give a complete overview of such a diverse field, we survey a selection of contemporary methods for the processing of discrete time signals with deep connections to matrix theory and numerical linear algebra. These topics include linear prediction, Wiener filtering, spectral estimation, and direction of arrival estimation. Notable omissions include classical methods of filter design, the fast Fourier transform (FFT), and filter structures. (See Chapter 58.)

64.1 Basic Concepts

We begin with basic definitions which are standard and can be found in [PM96].

Definitions:

A **signal** $\mathbf{x}$ is a real sequence with element k of the sequence for $k = 0, \pm 1, \pm 2, \dots$ given by $x(k)$. The set of all signals is a vector space with the sum $\mathbf{w} = \mathbf{x} + \mathbf{y}$ defined by $w(k) = x(k) + y(k)$ and the scalar product $\mathbf{v} = a\mathbf{x}$ where a is a real scalar given by $v(k) = ax(k)$.

A **linear time-invariant system** is a mapping $L(\mathbf{x}) = \mathbf{y}$ of an input signal $\mathbf{x}$ to an output signal $\mathbf{y}$ that satisfies the following properties:

1. Linearity: If $L(\mathbf{x}_0) = \mathbf{y}_0$ and $L(\mathbf{x}_1) = \mathbf{y}_1$, then $L(a\mathbf{x}_0 + b\mathbf{x}_1) = a\mathbf{y}_0 + b\mathbf{y}_1$ for any $a, b \in \mathbb{R}$.
2. Time-invariance: If $L(\mathbf{x}) = \mathbf{y}$ and $\hat{\mathbf{x}}$ is a shifted version of $\mathbf{x}$ given by $\hat{x}(k) = x(k - k_0)$, then $L(\hat{\mathbf{x}}) = \hat{\mathbf{y}}$ where $\hat{y}(k) = y(k - k_0)$. That is, shifted inputs lead to correspondingly shifted outputs.

The **impulse response** of a linear time-invariant system is the output $\mathbf{h}$ that results from applying as input the unit impulse δ where $\delta(0) = 1$ and $\delta(k) = 0$ for $k \neq 0$.

The **convolution** y of $\mathbf{h}$ and $\mathbf{x}$ is written $y = \mathbf{h} * \mathbf{x}$ and is defined by the sum

$$y(k) = \sum_{j=-\infty}^{\infty} h(j)x(k-j).$$

A signal $\mathbf{x}$ is **causal** if $x(k) = 0$ for $k < 0$.

A linear time-invariant system is **causal** if every causal input gives a causal output. An equivalent definition is that a system is **causal** if its impulse response sequence is causal.

The **z-transform** of a signal or impulse response $\mathbf{x}$ is

$$X(z) = \sum_{k=-\infty}^{\infty} x(k)z^{-k}$$

where $X(z)$ is defined in the region of convergence of the sum.

The **transfer function** of a linear system with impulse response $\mathbf{h}$ is the z -transform of the impulse response

$$H(z) = \sum_{k=-\infty}^{\infty} h(k)z^{-k}.$$

The **discrete time Fourier transform** of $\mathbf{x}$ is the function of ω obtained by evaluating the z -transform on the unit circle

$$X(e^{i\omega}) = \sum_{k=-\infty}^{\infty} x(k)e^{-i\omega k}.$$

If $\mathbf{x}$ is an impulse response of a system, then $X(e^{i\omega})$ is the **frequency response** of the system. The discrete time Fourier transform is defined only if the region of convergence of the z -transform includes the unit circle.

A filter is **minimum phase** if the zeros of the transfer function are in the unit circle.

A **finite impulse response** (FIR) filter is a system that maps an input signal $\mathbf{x}$ to an output signal y through the sum

$$y(k) = \sum_{j=0}^{n-1} b(j)x(k-j).$$

A rational **infinite impulse response** (IIR) filter is a causal system that maps inputs $\mathbf{x}$ to outputs y via a relation of the form

$$y(k) = \sum_{j=0}^{n-1} b(j)x(k-j) - \sum_{j=1}^m a(j)y(k-j).$$

Causality is an essential part of the definition; without this assumption the above relation does not uniquely define a mapping from inputs to outputs.

A signal $\mathbf{x}$ has **finite energy** if

$$\sum_{k=-\infty}^{\infty} |x(k)|^2 < \infty.$$

A system mapping input $\mathbf{x}$ to output y is **stable** if $|x(k)| < B_x$ for $0 < B_x < \infty$ and for all k implies that there is $0 < B_y < \infty$ such that the output satisfies $|y(k)| < B_y$ for all k .

Facts:

1. The impulse response $\mathbf{h}$ uniquely determines a linear time-invariant system. The mapping from the input $\mathbf{x}$ to the output $\mathbf{y}$ is given by $\mathbf{y} = \mathbf{h} * \mathbf{x}$. Convolution is commutative so that if $\mathbf{y} = \mathbf{g} * (\mathbf{h} * \mathbf{x})$, then $\mathbf{y} = \mathbf{h} * (\mathbf{g} * \mathbf{x})$. Thus, the order in which two linear time-invariant filters are applied to a signal does not matter.
2. A causal system can be applied in real time in the sense that output $y(k)$ can be computed from the convolution $\mathbf{y} = \mathbf{h} * \mathbf{x}$ as soon as $x(k)$ is available.
3. A system defined by its impulse response $\mathbf{h}$ is stable if and only if

$$\sum_{k=-\infty}^{\infty} |h(k)| < \infty.$$

4. An expression for the z -transform of a signal (or an impulse response) does not uniquely determine the signal. It is also necessary to know the region of convergence. If $X(z)$ is the z -transform of $\mathbf{x}$, then the inverse transform is

$$x(k) = \frac{1}{2\pi i} \oint X(z) z^{k-1} dz$$

where the integral is taken over any contour enclosing the origin and in the region where $X(z)$ is analytic. A system is uniquely determined by its transfer function and the region of convergence of the transfer function.

5. An FIR filter has impulse response

$$h(k) = \begin{cases} b(k) & \text{for } 0 \leq k \leq n-1 \\ 0 & \text{otherwise.} \end{cases}$$

6. An FIR filter has transfer function

$$H(z) = b(0) + b(1)z^{-1} + \cdots + b(n-1)z^{-(n-1)}.$$

7. The impulse response $\mathbf{h}$ of a causal IIR filter is the unique solution to the recurrence

$$h(k) = \begin{cases} 0 & k \leq 0 \\ b(k) - \sum_{j=1}^m a(j)h(k-j) & 0 \leq k \leq n-1 \\ -\sum_{j=1}^m a(j)h(k-j) & k \geq n \end{cases}.$$

This recurrence uniquely determines a causal impulse response $\mathbf{h}$ and, hence, uniquely determines the mapping from inputs to outputs.

8. A rational IIR filter has transfer function

$$H(z) = \frac{b(0) + b(1)z^{-1} + \cdots + b(n-1)z^{-(n-1)}}{1 + a(1)z^{-1} + \cdots + a(m)z^{-m}}.$$

9. A linear time-invariant system with input $\mathbf{x}$ and impulse response $\mathbf{h}$ with z -transform $X(z)$ and transfer function $H(z)$ has output $\mathbf{y}$ with z -transform

$$Y(z) = H(z)X(z).$$

10. A discrete version of the Paley–Wiener theorem states that $\mathbf{x}$ is causal and has finite energy if and only if the corresponding z -transform $X(z)$ is analytic in the exterior of the unit circle and

$$\sup_{1 < r < \infty} \int_{-\pi}^{\pi} |X(re^{i\omega})|^2 d\omega < \infty.$$

If $|X(e^{i\omega})|$ is square integrable, then $|X(e^{i\omega})|$ is the Fourier transform of some causal system finite energy signal if and only if

$$\int_{-\pi}^{\pi} |\ln(|X(e^{i\omega})|)| d\omega < \infty.$$

11. For rational transfer functions $H(z)$ corresponds to a causal stable system if and only if the poles of $H(z)$ are in the unit circle. Thus, FIR filters are always stable and IIR filters are stable when the poles of the transfer function are in the unit circle.
12. A filter that is minimum phase has a causal stable inverse filter with transfer function $G(z) = 1/H(z)$. That is, filtering a signal $\mathbf{x}$ by a filter with z -transform $H(z)$ and then by a filter with z -transform $G(z)$ gives an output with z -transform $G(z)H(z)X(z) = X(z)$. The minimum phase property is of particular significance since the only filters that can be applied in real time and without excessive growth of errors due to noise are those that are causal and stable. In most circumstances only minimum phase filters are invertible in practice.

Examples:

1. The causal impulse response $\mathbf{h}$ with $h(k) = a^k$ for $k \geq 0$ and $h(k) = 0$ for $k < 0$ has z -transform $H(z) = 1/(1 - az^{-1})$ with region of convergence $|z| > |a|$. The anticausal impulse response $\hat{\mathbf{h}}$ with $\hat{h}(k) = -a^k$ for $k < 0$ and $\hat{h}(k) = 0$ for $k \geq 0$ has z -transform $H(z) = 1/(1 - az^{-1})$ with region of convergence $|z| < |a|$.
2. The causal impulse response $\mathbf{h}$ with

$$h(k) = \begin{cases} 2^{-k} + 3^{-k} & \text{for } k \geq 0 \\ 0 & k < 0 \end{cases}$$

has z -transform

$$H(z) = \frac{1}{1 - (2z)^{-1}} + \frac{1}{1 - (3z)^{-1}} = \frac{2 - \frac{5}{6}z^{-1}}{1 - \frac{5}{6}z^{-1} + \frac{1}{6}z^{-2}}$$

with region of convergence $|z| > 1/2$. The impulse response can be realized by a rational IIR filter of the form

$$y(k) = 2x(k) - \frac{5}{6}x(k-1) + \frac{5}{6}y(k-1) + \frac{1}{6}y(k-2).$$

The zeros of $z^2 - (5/6)z + (1/6)$ are $1/2$ and $1/3$ so that the system is stable.

64.2 Random Signals

Definitions of statistical terms can be found in Chapter 52.

Definitions:

A **random signal** or **random process** $\mathbf{x}$ is a sequence $x(k)$ of real random variables indexed by $k = 0, \pm 1, \pm 2, \dots$

A random signal $\mathbf{x}$ is **wide-sense stationary** (for brevity, we will use **stationary** to mean the same thing) if the mean $\mu_x = E[x(k)]$ and **autocorrelation sequence** $r_{xx}(k) = E[x(j)x(j-k)]$ do not depend on j . We assume that all random signals are wide-sense stationary. Two stationary random signals $\mathbf{x}$ and $\mathbf{y}$ are jointly stationary if the **cross correlation** sequence $r_{yx}(k) = E[y(j)x(j-k)]$ does not depend on j . When referring to two stationary signals, we always assume that the signals are jointly stationary as well.

A sequence $\mathbf{x}(k)$ of real random $n \times 1$ vectors indexed by $k = 0, \pm 1, \pm 2, \dots$ is **stationary** if $E[\mathbf{x}(k)]$ and $E[\mathbf{x}(k)\mathbf{x}^T(k)]$ do not depend on k .

The **autocorrelation matrix** of a sequence of stationary random vectors $\mathbf{x}(k)$ is

$$R_{\mathbf{x}} = E[\mathbf{x}(k)\mathbf{x}^T(k)].$$

A zero mean stationary random signal n is a **white noise process** if it has autocorrelation sequence

$$r_{nn}(j) = E[n(k)n(k-j)] = \begin{cases} \sigma_n^2 & j = 0 \\ 0 & j \neq 0 \end{cases}$$

where σ_n^2 is the **variance** of n .

A white noise driven **autoregressive process** (AR process) x is of the form

$$x(k) = n(k) - \sum_{j=1}^n a(j)x(k-j)$$

where n is a white noise process and the filtering of n to obtain x is causal.

Given a signal x , we denote the z -transform of the autocorrelation sequence of x by $S_x(z)$. The **spectral density** of a stationary random signal x with autocorrelation sequence $r_{xx}(k)$ is the function of ω given by evaluating $S_x(z)$ on the unit circle

$$S_x(e^{i\omega}) = \sum_{j=-\infty}^{\infty} r_{xx}(j)e^{-ij\omega}.$$

A random signal with spectral density $S_x(e^{i\omega})$ is **bandlimited** if $S_x(e^{i\omega}) = 0$ on a subset of $[-\pi, \pi]$ of nonzero measure.

A random signal with spectral density $S_x(e^{i\omega})$ has **finite power** if

$$\int_{-\pi}^{\pi} S_x(e^{i\omega}) d\omega < \infty.$$

The **spectral factorization**, when it exists, of a spectral density $S_x(z)$ is a factorization of the form

$$S_x(z) = L(z)\overline{L(z^{-1})}$$

where $L(z)$ and $1/L(z)$ are both analytic for $|z| > 1$. Interpreted as transfer functions $L(z)$ and $1/L(z)$ both describe causal, stable systems. That is, $L(z)$ is causal and stable and causally and stably invertible.

Facts:

1. If $\mathbf{x}(k)$ is a sequence of vectors obtained by sampling a random sequence of vectors, then under suitable ergodicity assumptions

$$R_{\mathbf{x}} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=0}^n \mathbf{x}(k)\mathbf{x}^T(k).$$

In computational practice $R_{\mathbf{x}}$ is often estimated using a truncated sum in which n is finite.

2. The expectation $\langle x, y \rangle = E[xy]$ can be viewed as an inner product on a real Hilbert space of zero mean random variables with $E[|x|^2] < \infty$. This gives many results in optimal filtering a geometric interpretation in terms of orthogonalization and projection on the span of a set of random variables in a way that is perfectly analogous to least squares problems in any other Hilbert space. From this point of view, a white noise process n is a sequence of orthogonal random variables.

3. If a random signal $\mathbf{x}$ with autocorrelation sequence $\mathbf{r}_x$ is passed through a system with impulse response $\mathbf{h}$, then the output $\mathbf{y}$ has autocorrelation

$$\mathbf{r}_y = \mathbf{h} * \mathbf{r}_x * \tilde{\mathbf{h}}$$

where $\tilde{\mathbf{h}}$ is defined by $\tilde{h}(j) = h(-j)$.

4. The spectral density is nonnegative, $S_x(e^{i\omega}) \geq 0$.
 5. The spectral density of white noise is $S_n(e^{i\omega}) = \sigma_n^2$.
 6. [Pap85] If a random signal $\mathbf{x}$ with spectral density $S_x(z)$ passes through a filter with transfer function $H(z)$, then the output $\mathbf{y}$ is a stationary signal with

$$S_y(z) = S_x(z)H(z)\overline{H(z^{-1})} \quad \text{or} \quad S_y(e^{i\omega}) = S_x(e^{i\omega})|H(e^{i\omega})|^2.$$

7. If $L(z)$ is the spectral factor of $S_x(z)$, then on the unit circle we have $S_x(e^{i\omega}) = |L(e^{i\omega})|^2$.
 8. [SK01], [GS84] The spectral factorization exists if and only if the signal has finite power and satisfies

$$\int_{-\pi}^{\pi} \ln(S_x(e^{i\omega})) d\omega > -\infty.$$

If $S_x(e^{i\omega})$ is zero on a set of nonzero measure, then the condition is not satisfied. Thus, bandlimited signals do not have a spectral factorization.

9. If a spectral factor $L(z)$ exists for a signal $\mathbf{x}$ with spectral density $S_x(e^{i\omega})$ and

$$L(z) = \sum_{k=0}^{\infty} l(k)z^{-k} \quad \text{and} \quad M(z) = \frac{1}{L(z)} = \sum_{k=0}^{\infty} m(k)z^{-k},$$

then the filtered signal $\mathbf{n} = \mathbf{m} * \mathbf{x}$ given by

$$n(k) = \sum_{j=0}^{\infty} m(j)x(k-j)$$

is a white noise process with $S_n(e^{i\omega}) = 1$. The original signal can be obtained by causally and stably filtering $\mathbf{n}$ with $\mathbf{l}$:

$$x(k) = \sum_{j=0}^{\infty} l(j)n(k-j).$$

The signal $\mathbf{n}$ is the **innovations** process for $\mathbf{x}$ and the representation of $\mathbf{x}$ through filtering $\mathbf{n}$ by $\mathbf{l}$ is the **innovations representation** for $\mathbf{x}$. It is a model of $\mathbf{x}$ as a filter driven by white noise. The innovations representation is an orthogonalization of the sequence of random variables $\mathbf{x}$. The innovations representation for $\mathbf{x}$ is analogous to the QR factorization of a matrix.

Examples:

1. For a white noise process $\mathbf{n}$ with variance $\sigma_n^2 = 1$, define the autoregressive process $\mathbf{x}$ by

$$x(k) = n(k) + ax(k-1)$$

where $|a| < 1$. The impulse response and transfer function of the IIR filter generating $\mathbf{x}$ are

$$h(k) = \begin{cases} 0 & k < 0 \\ a^k & k \geq 0 \end{cases}, \quad \text{and} \quad H(z) = \frac{1}{1 - az^{-1}}.$$

The autocorrelation sequence for $\mathbf{x}$ is the sequence r_{xx} given by

$$r_{xx}(k) = \frac{1}{1 - a^2} a^{|k|}.$$

The autocorrelation sequence has z -transform

$$S_x(z) = \frac{1}{(1 - az^{-1})(1 - az)}$$

so that the spectral density is

$$S_x(e^{i\omega}) = \frac{1}{1 + a^2 - 2a \cos(\omega)}.$$

The spectral factor is

$$L(z) = \frac{1}{1 - az^{-1}}.$$

64.3 Linear Prediction

Definitions:

The **linear prediction** problem is to find an estimate $\hat{\mathbf{x}}$ of a stationary random signal $\mathbf{x}$ as a linear combination

$$\hat{x}(k) = \sum_{j=1}^{\infty} a(j)x(k-j),$$

with the $a(j)$ chosen to minimize the **mean square prediction error** $E_{\infty} = E[|e(k)|^2]$ where $e(k) = \hat{x}(k) - x(k)$ is the **prediction error**. Thus, the goal is to find a current estimate $\hat{x}(k)$ of $x(k)$ from the past values $x(k-1), x(k-2), \dots$

A random signal is **predictable** if $E_{\infty} = 0$. A signal that is not predictable is **regular**.

The **order n linear prediction problem** is to find an estimate of a stationary random signal $\mathbf{x}$ with a signal $\hat{\mathbf{x}}$ of the form $\hat{x}(k) = \sum_{j=1}^n a_n(j)x(k-j)$ so as to minimize the mean square error $E_n = E[|e(k)|^2]$ where $e(k) = \hat{x}(k) - x(k)$.

A random signal is **order n predictable** if $E_n = 0$.

The **prediction error filter** and the **order n prediction error filter** are the filters with transfer functions

$$E(z) = 1 - \sum_{j=1}^{\infty} a(j)z^{-j}$$

and

$$E_n(z) = 1 - \sum_{j=1}^n a_n(j)z^{-j}.$$

The **Wiener–Hopf equations** for the optimal prediction coefficients $a(j)$ are

$$r_{xx}(l) = \sum_{j=1}^{\infty} a(j)r_{xx}(l-j)$$

for $l \geq 1$.

Given a random signal $x(k)$ with autocorrelation $r_{xx}(k)$, the optimal prediction coefficients $a^{(n)}(j)$ and mean square prediction error E_n that satisfy the **Yule–Walker equations** are

$$\begin{bmatrix} r_{xx}(0) & r_{xx}(1) & \cdots & r_{xx}(n) \\ r_{xx}(1) & r_{xx}(0) & \ddots & \vdots \\ \vdots & \ddots & \ddots & r_{xx}(1) \\ r_{xx}(n) & \cdots & r_{xx}(1) & r_{xx}(0) \end{bmatrix} \begin{bmatrix} 1 \\ -a_n(1) \\ \vdots \\ -a_n(n) \end{bmatrix} = \begin{bmatrix} E_n \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

or $T_n \mathbf{a}_n = E_n \mathbf{e}_1$.

The **Levinson–Durbin algorithm** for the computation solution of the order n linear prediction problem is

$$\mathbf{a}_n = \begin{bmatrix} \mathbf{a}_{n-1} \\ 0 \end{bmatrix} + \gamma_n \begin{bmatrix} 0 \\ R\mathbf{a}_{n-1} \end{bmatrix}$$

where R is the permutation that reverses the order of the elements of $\mathbf{a}_{n-1}$. The quantity γ_n is computed from the relation

$$\gamma_n = -\frac{r_{xx}(n) - \sum_{j=1}^{n-1} a_{n-1}(j)r_{xx}(n-j)}{E_{n-1}}$$

and the mean square prediction error is computed from $E_n = E_{n-1}(1 - |\gamma_n|^2)$. The algorithm starts with $E_0 = r_{xx}(0)$ and $\mathbf{a}_0 = 1$. If $x(k)$ is order n predictable, then the process terminates with the order n predictor and $E_n = 0$.

The parameters γ_k are known as **reflection coefficients**, **partial correlation coefficients** or **Schur parameters**. The **complementary parameters** are $\delta_k = \sqrt{1 - |\gamma_k|^2}$.

Facts:

1. [Pap85] The Wold decomposition theorem: Every stationary random signal can be represented as the sum of a regular and a predictable signal

$$x(k) = x_r(k) + x_p(k).$$

The two components are orthogonal with $E[x_r(k)x_p(k)] = 0$.

2. [Pap85] The Wiener–Hopf equations can be interpreted as stating that the prediction error $\hat{x}(k) - x(k)$ is orthogonal to $x(k-l)$ for $l \geq 1$ in the inner product $\langle x, y \rangle = E[xy]$.
3. The Toeplitz matrix T_n in the Yule–Walker equations is positive semidefinite.
4. The Levinson–Durbin algorithm is an $O(n^2)$ order recursive method for solving the linear prediction problem. In most cases the numerical results from applying the algorithm are comparable to the numerically stable Cholesky factorization procedure [Cyb80]. An alternative fast method, the Schur algorithm, computes a Cholesky factorization and can be proven to be numerically stable [BBH95].
5. The following are equivalent:
 - (a) T_{n-1} is positive definite and T_n is positive semidefinite.
 - (b) $|\gamma_j| < 1$ for $1 \leq j \leq n-1$ and $|\gamma_n| = 1$.
 - (c) $E_j > 0$ for $1 \leq j \leq n-1$ and $E_n = 0$.
 - (d) The polynomial $z^n E_n(z)$ has all zeros on the unit circle and each polynomial $z^k E_k(z)$ for $k = 1, 2, \dots, n-1$ has all zeros strictly inside the unit circle.
6. [Pap85] The prediction errors are monotonic: $E_k \geq E_{k+1}$ and $E_k \rightarrow E_\infty \geq 0$ as $k \rightarrow \infty$.

7. [SK01] Let

$$a_k(z) = 1 - a_k(1)z - a_k(2)z^2 - \cdots - a_k(k)z^k$$

where $\overline{a_k(j)}$ are the prediction coefficients for a signal $\mathbf{x}$ with spectral density $S_x(z)$. If $S_x(z) = L(z)L(\overline{z^{-1}})$ is the spectral factorization of $S_x(z)$, then

$$L(z) = \lim_{k \rightarrow \infty} \frac{E_k^{1/2}}{r_{xx}(0)} \frac{1}{a_k(\overline{z^{-1}})}.$$

A survey, including other representations of the spectral factor and algorithms for computing it, is given in [SK01].

8. [GS84] A finite power random signal $\mathbf{x}$ is predictable (i.e., $E_\infty = 0$) if and only if

$$\int_{-\pi}^{\pi} \ln(S_x(e^{i\omega})) d\omega > -\infty.$$

If the signal is regular, then we have the Kolmogorov–Szegő error formula

$$E_\infty = \exp \left(\int_{-\pi}^{\pi} \ln S(e^{i\omega}) d\omega \right) > 0.$$

Examples:

1. Consider the autoregressive process $\mathbf{x}$ given by

$$x(k) = n(k) + ax(k-1)$$

where $\mathbf{n}$ is a white noise process with $\sigma_n^2 = 1$. The solution to the order 1 linear prediction problem is given by the Yule–Walker system

$$\begin{bmatrix} \frac{1}{1-a^2} & \frac{a}{1-a^2} \\ \frac{a}{1-a^2} & \frac{1}{1-a^2} \end{bmatrix} \begin{bmatrix} 1 \\ -a_1(1) \end{bmatrix} = \begin{bmatrix} E_1 \\ 0 \end{bmatrix}.$$

The Levinson–Durbin algorithm starts with $E_0 = r_{xx}(0) = 1/(1-a^2)$ and $\mathbf{a}_0 = 1$. Thus,

$$\gamma_1 = -\frac{r_{xx}(1)}{E_0} = -\frac{a/(1-a^2)}{1/(1-a^2)} = -a$$

so that

$$\begin{bmatrix} 1 \\ -a_1(1) \end{bmatrix} = \mathbf{a}_1 = \begin{bmatrix} \mathbf{a}_0 \\ 0 \end{bmatrix} + \gamma_1 \begin{bmatrix} 0 \\ \mathbf{a}_0 \end{bmatrix} = \begin{bmatrix} 1 \\ -a \end{bmatrix}.$$

Thus, $a_1(1) = a$ and the optimal linear prediction of $x(k)$ from $x(k-1)$ is given by

$$\hat{x}(k) = ax(k-1).$$

The prediction error is given by

$$E_1 = E[\hat{x}(k) - x(k)] = E[n(k)] = 1.$$

64.4 Wiener Filtering

Linear optimal filtering of stationary signals was introduced by Wiener [Wie49] and Kolmogorov [Kol41]. The material here is covered in books on adaptive filtering [Hay91] and linear estimation [KSH00].

Definitions:

The general **discrete Wiener filtering problem** is the following: Given stationary signals $\mathbf{x}$ and $\mathbf{y}$ find a filter with impulse response $\mathbf{w}$ such that

$$E_w = E \left[\left| y(k) - \sum_{j=-\infty}^{\infty} w(j)x(k-j) \right|^2 \right]$$

is minimal. The filter with impulse response $\mathbf{w}$ minimizing E_w is the **Wiener filter**. The goal is to approximate a desired signal $\mathbf{y}$ by filtering the signal $\mathbf{x}$. Equivalently, we seek the best approximation to $y(k)$ in the span of the $x(j)$ for $-\infty < j < \infty$. Note that we leave open the possibility that the Wiener filter is not causal.

Depending on specific relations that may be imposed on $\mathbf{y}$ and $\mathbf{x}$, the Wiener filtering problem specializes in several important ways: The **Wiener prediction problem** for which $y(k) = x(k+l)$ where $l \geq 1$ and $\mathbf{w}$ is assumed causal; the **Wiener smoothing problem** for which $\mathbf{y}$ is arbitrary and $\mathbf{x}$ is a noise corrupted version of $\mathbf{y}$ given by $\mathbf{x} = \mathbf{y} + \mathbf{n}$ where $\mathbf{n}$ is the noise; and the **Wiener deconvolution problem** for which we have arbitrary desired signals $\mathbf{y}$ and $\mathbf{x}$ obtained from $\mathbf{y}$ by convolution and the addition of noise, $\mathbf{x} = \mathbf{h} * \mathbf{y} + \mathbf{n}$. For the Wiener prediction problem, the goal is to predict future values of $\mathbf{x}$ from past values. For the Wiener smoothing problem, the goal is to recover an approximation to a signal $\mathbf{y}$ from a noise corrupted version of the signal $\mathbf{x}$. For the Wiener deconvolution, the goal is to invert a filter in the presence of noise to obtain the signal $\mathbf{y}$ from the filtered and noise corrupted version $\mathbf{x}$.

The **FIR Wiener filtering problem** is to choose an FIR filter with impulse response $w(k)$ such that

$$E \left[\left| y(k) - \sum_{j=0}^{n-1} w(j)x(k-j) \right|^2 \right]$$

is minimal.

The **causal part** of a signal $x(k)$ is

$$[x(k)]_+ = \begin{cases} x(k) & \text{if } k \geq 0 \\ 0 & \text{if } k < 0 \end{cases} \quad \text{or} \quad [X(z)]_+ = \sum_{k=0}^{\infty} x(k)z^{-k}.$$

Facts:

1. The stationarity assumptions ensure that the coefficients $w(j)$ that are used to estimate $y(k)$ from the signal $\mathbf{x}$ do not depend on k . That is, the coefficients $w(j)$ that minimize E_w also minimize

$$E \left[\left| y(k-k_0) - \sum_{j=0}^{n-1} w(j)x(k-k_0-j) \right|^2 \right]$$

so that the Wiener filter is a linear time invariant system.

2. Let $S_x(z) = L(z)L(\overline{z^{-1}})$,

$$r_{yx}(k) = E[y(j)x(j-k)], \quad \text{and} \quad S_{yx}(e^{i\omega}) = \sum_{k=-\infty}^{\infty} r_{yx}(k)e^{ik\omega}.$$

If we do not require causality, then the Wiener filter has z -transform

$$W(z) = \frac{S_{yx}(z)}{S_{xx}(z)}.$$

If we do impose causality, then the Wiener filter is

$$\left[\frac{S_{yx}(z)}{L^*(z^{-*})} \right]_+ \frac{1}{L(z)}.$$

3. As seen above the Wiener filter depends on the cross correlations $r_{yx}(k)$, but not on knowledge of the elements of the unknown sequence $y(k)$. Thus, it is possible to estimate the signal y by filtering x without direct knowledge of y . If the autocorrelation sequences $r_{xx}(k)$ and $r_{nn}(k)$ are available (or can be estimated), then $r_{yx}(k)$ can often be computed in a straightforward way. For the Wiener prediction problem, $r_{yx}(k) = r_{xx}(l + k)$. If the noise n and signal y are uncorrelated so that $r_{ny}(k) = 0$ for all k , then for the Wiener smoothing problem $r_{yx} = r_{xx} - r_{nn}$. For the Wiener deconvolution problem, if g is the inverse filter of h and if y and n are uncorrelated, then $r_{yx} = g * (r_{xx} - r_{nn})$.
4. The coefficients for an FIR Wiener filter satisfy the Wiener–Hopf equation

$$\begin{bmatrix} r_{xx}(0) & r_{xx}(-1) & \cdots & r_{xx}(-m+1) \\ r_{xx}(1) & r_{xx}(0) & \ddots & \vdots \\ \vdots & \ddots & \ddots & r_{xx}(-1) \\ r_{xx}(m-1) & \cdots & r_{xx}(1) & r_{xx}(0) \end{bmatrix} \begin{bmatrix} w(0) \\ w(1) \\ \vdots \\ w(m-1) \end{bmatrix} = \begin{bmatrix} r_{yx}(0) \\ r_{yx}(1) \\ \vdots \\ r_{yx}(m-1) \end{bmatrix}.$$

As with linear prediction, this system of equations can be solved using fast algorithms for Toeplitz systems.

Examples:

1. We consider the first order FIR Wiener prediction problem of predicting $y(k) = x(k+2)$ from the autoregressive process x with $x(k) = n(k) + ax(k-1)$. Thus, we seek $w(0)$ such that

$$E [|x(k+2) - w(0)x(k)|^2]$$

is minimal. Since

$$r_{xx}(0) = \frac{1}{1-a^2}$$

and

$$r_{yx}(0) = r_{xx}(2) = \frac{a^2}{1-a^2}$$

the Wiener–Hopf equation for $w(0)$ is

$$\frac{1}{1-a^2} w(0) = \frac{a^2}{1-a^2}.$$

64.5 Adaptive Filtering

Definitions:

An FIR **adaptive filter** is a filter with coefficients that are updated to match the possibly changing statistics of the input $\mathbf{x}$ and the desired output $\mathbf{y}$. The output is

$$\hat{y}(k) = \sum_{j=0}^{m-1} w_k(j)x(k-j)$$

where the $w_k(j)$ depend on k . The goal is similar to that of the Wiener filter, that is, to approximate the sequence $\mathbf{y}$. The difference is that instead of computing a fixed set of coefficients $w(j)$ from knowledge of the statistics of stationary signals $\mathbf{x}$ and $\mathbf{y}$, the coefficients are allowed to vary with k and are computed from actual samples of the sequences $\mathbf{x}$ and $\mathbf{y}$.

A **recursive least squares (RLS)** adaptive FIR filter is a rule for updating the coefficients $w_k(j)$. The coefficients are chosen to minimize

$$\min_{\mathbf{w}_k} \left\| D_{\rho,k} \begin{bmatrix} x(l) & x(l-1) & \cdots & x(l-m+1) \\ x(l+1) & x(l) & \cdots & x(l-m+2) \\ \vdots & \ddots & \ddots & \vdots \\ x(k) & x(k-1) & \cdots & x(k-m+1) \end{bmatrix} \begin{bmatrix} w_k(0) \\ w_k(1) \\ \vdots \\ w_k(m-1) \end{bmatrix} - D_{\rho,k} \begin{bmatrix} y(l) \\ y(l+1) \\ \vdots \\ y(k) \end{bmatrix} \right\|_2$$

where $k \geq l + m - 1$ and $D_{\rho,k} = \text{diag}(\rho^{k-l}, \rho^{k-l-1}, \dots, \rho^0)$ with $0 < \rho \leq 1$. Equivalently, the vector $\mathbf{w}_k$ is chosen to solve $\min_{\mathbf{w}_k} \|D_{\rho,k} T_{l,k} \mathbf{w}_k - D_{\rho,k} \mathbf{y}_k\|$. The parameter ρ is chosen to decrease the influence of older data. This is of use when the signals $\mathbf{x}$ and $\mathbf{y}$ are only approximately stationary, i.e., their statistical properties are slowly time varying.

A signal $\mathbf{x}$ is **persistently exciting of order m** if there exist $c_1 > 0$, $c_2 \geq c_1$, and j such that

$$c_1 I \leq \lambda_p \left(\frac{1}{k-l+1} T_{l,k}^T T_{l,k} \right) \leq c_2 I$$

for each eigenvalue λ_p of $T_{l,k}^T T_{l,k} / (k-l+1)$ and for all $k \geq j$.

Facts:

1. The vector $\mathbf{w}_k$ can be updated using using the following recursive formulas:

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{g}_k (y(k) - \mathbf{w}_{k-1}^T \mathbf{x}_k)$$

where

$$\mathbf{x}_k = \begin{bmatrix} x(k) \\ \vdots \\ x(k-m+1) \end{bmatrix},$$

$$\mathbf{g}_k = \frac{\rho^{-2} P_{k-1} \mathbf{x}_k}{1 + \rho^{-2} \mathbf{x}_k^T P_{k-1} \mathbf{x}_k},$$

with

$$P_k = \rho^{-2} P_{k-1} - \rho^{-2} \mathbf{g}_k \mathbf{x}_k^T P_{k-1}.$$

With initialization $P_k = (T_{l,k}^T D_{\rho,k}^2 T_{l,k})^{-1}$ and $\mathbf{w}_k = P_k T_{l,k}^T D_{\rho,k}^2 \mathbf{y}_k$, the recurrences compute the solution $\mathbf{w}_k$ to the recursive least squares problem.

2. [GV96] The weight vector $\mathbf{w}_k$ can also be updated as k increases with an $O(n^2)$ cost using a standard QR updating algorithm. Such algorithms have good numerical properties.
3. If $\mathbf{x}$ and $\mathbf{y}$ are stationary, if $\rho = 1$, and if $\mathbf{x}$ is persistently exciting, then $\mathbf{w}_k$ converges to the Wiener filter as $k \rightarrow \infty$. Under these assumptions the effect of initializing P_k incorrectly becomes negligible as $k \rightarrow \infty$. Thus, convergence does not depend on the correct initialization of $P_k = T_{l,k}^T T_{l,k}$. For simplicity, it is common to choose an initial P_k that is a multiple of the identity.
4. If $\mathbf{x}$ and $\mathbf{y}$ are not stationary, then ρ is typically chosen to be less than one in order to allow the filter to adapt to changing statistics. If the signals really are stationary, then choice of $\rho < 1$ sacrifices convergence of the filter.
5. There are a variety of algorithms that exploit the Toeplitz structure of the least squares problem, reducing the cost per update of $\mathbf{w}_k$ to $O(n)$ operations [CK84]. An algorithm that behaves well in finite precision is analyzed in [Reg93].

Examples:

1. Consider applying a recursive least squares adaptive filter to signals $\mathbf{x}$ and $\mathbf{y}$ with

$$x(1) = 1, x(2) = 2, x(3) = 3, x(4) = 4$$

and

$$y(2) = 5, y(3) = 6, y(4) = 8.$$

The vector of filter coefficients $\mathbf{w}_3$ satisfies

$$\begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix} \mathbf{w}_3 = \begin{bmatrix} 5 \\ 6 \end{bmatrix},$$

so that

$$\mathbf{w}_3 = \begin{bmatrix} 4 \\ -3 \end{bmatrix}.$$

We also have

$$P_3 = \left(\begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix}^T \begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix} \right)^{-1} = \begin{bmatrix} 13 & 8 \\ 8 & 5 \end{bmatrix},$$

so that

$$\mathbf{g}_4 = \frac{P_3 \mathbf{x}_4}{1 + \mathbf{x}_4^T P_3 \mathbf{x}_4} = \frac{1}{6} \begin{bmatrix} -4 \\ 7 \end{bmatrix}.$$

Thus, the next coefficient vector $\mathbf{w}_4$ is

$$\mathbf{w}_4 = \mathbf{w}_3 + \mathbf{g}_3 (y(4) - \mathbf{w}_3^T \mathbf{x}_4) = \begin{bmatrix} 10/3 \\ -11/6 \end{bmatrix}.$$

The process can be continued by updating P_4 , $\mathbf{g}_5$, and $\mathbf{w}_5$.

64.6 Spectral Estimation

Definitions:

The **deterministic spectral estimation** problem is to estimate the spectral density $S_x(e^{i\omega})$ from a finite number of correlations $r_{xx}(k)$ for $0 \leq k \leq n$. Note that this definition is not taken to imply that the signal $\mathbf{x}$ is deterministic, but that the given data are correlations rather than samples of the random signals.

The **stochastic spectral estimation** problem is to estimate $S(e^{i\omega})$ from samples of the random signal $x(k)$ for $0 \leq k \leq n$.

The problem of estimating **sinusoids in noise** is to find the frequencies ω_j of

$$\hat{x}(k) = \alpha_1 \cos(\omega_1 k + \eta_1) + \alpha_2 \cos(\omega_2 k + \eta_2) + \cdots + \alpha_p \cos(\omega_p k + \eta_p) + w(k) = x(k) + n(k)$$

where $\mathbf{n}$ is zero mean white noise with variance σ_n^2 and $\mathbf{x}$ is the exact sinusoidal signal with random frequencies $\omega_j \in [0, 2\pi)$ and phases η_j . The frequencies and phases are assumed to be uncorrelated with the noise $\mathbf{n}$. The available data can be the samples of the signal $\mathbf{x}$ or the autocorrelation sequence $r_{xx}(j)$.

Facts:

- [PM96] A traditional method for the deterministic spectral estimation problem is to assume that any $r_{xx}(j)$ not given are equal to zero. When additional information is available in the form of a signal model, it is possible to do better by extending the autocorrelation sequence in a way that is consistent with the model.
- [Pap85] The method of maximum entropy: The sequence $r_{xx}(k)$ is extended so as to maximize the entropy rate

$$\int_{-\pi}^{\pi} \ln S(e^{i\omega}) d\omega,$$

where it is assumed that the random signal $\mathbf{x}$ is a sequence of normal random variables. It can be shown that the spectrum that achieves this is the spectrum of an autoregressive signal

$$S(e^{i\omega}) = \frac{E_n}{\left| 1 - \sum_{j=1}^n a_n(j) e^{-i\omega j} \right|^2},$$

where the $a_n(j)$ are the coefficients obtained from the Yule–Walker equations.

- [Pis73] Pisarenko's method: Given a signal comprised of sinusoids in noise, if $\sigma_n = 0$, then the signal sequence $\mathbf{x}$ is order p predictable and the signal and its autocorrelation sequence $\mathbf{r}_{xx}$ satisfy

$$\begin{aligned} x(k) &= a(1)x(k-1) + a(2)x(k-2) + \cdots + a(p)x(k-p), \\ r_{xx}(k) &= a(1)r_{xx}(k-1) + a(2)r_{xx}(k-2) + \cdots + a(p)r_{xx}(k-p), \end{aligned}$$

where $\mathbf{a}_p = [1 \quad -a_p(1) \quad \cdots \quad -a_p(p)]^T$ is a null vector of the $(p+1) \times (p+1)$ autocorrelation matrix $T = [t_{ij}] = [r_{xx}(i-j)]$. This is the Yule–Walker system with $n = p$ and $E_p = 0$. When $\sigma_n \neq 0$, we have $\hat{T} = T + \sigma_n^2 I$ where $\hat{T}$ is the autocorrelation matrix for $\hat{x}(k)$. The value of σ_n^2 is the smallest eigenvalue of $\hat{T}$. Given knowledge of σ_n , T can be computed from $T = \hat{T} - \sigma_n^2 I$. The prediction coefficients can then be computed from the Yule–Walker equation $T\mathbf{a}_p = 0$. The zeros of the prediction error filter

$$z^p E_p(z) = z^p \left(1 - \sum_{j=1}^p a(j) z^{-j} \right)$$

all lie on the unit circle at $e^{i\omega_j}$ for $j = 1, 2, \dots, p$, giving the frequencies of the sinusoids.

4. [AGR87] The zeros of a polynomial are often sensitive to small errors in the coefficients. Computing such zeros accurately can be difficult. As an alternative the zeros of $z^p E_p(z)$ can be found as the eigenvalues of a unitary Hessenberg matrix. Let γ_j and δ_j be the Schur parameters and complementary parameters for the autocorrelation matrix R . Then $|\gamma_j| < 1$ for $1 \leq j \leq p-1$ and $|\gamma_p| = 1$. The matrix

$$H = G_1(\gamma_1)G_2(\gamma_2) \cdots G_p(\gamma_p)$$

with

$$G_j(\gamma_j) = I_{j-1} \oplus \begin{bmatrix} -\gamma_j & \delta_j \\ \delta_j & \overline{\gamma}_j \end{bmatrix} \oplus I_{p-j-1}$$

is unitary and upper Hessenberg. An explicit formula for the elements of H is

$$H = \begin{bmatrix} -\gamma_1 & -\delta_1\gamma_2 & -\delta_1\delta_2\gamma_3 & \cdots & -\delta_1 \cdots \delta_{p-1}\gamma_p \\ \delta_1 & -\overline{\gamma}_1\gamma_2 & -\overline{\gamma}_1\delta_2\gamma_3 & \cdots & -\overline{\gamma}_1\delta_2 \cdots \delta_{p-1}\gamma_p \\ & \delta_2 & -\overline{\gamma}_2\gamma_3 & & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & \delta_{p-1} & -\overline{\gamma}_{p-1}\gamma_p \end{bmatrix}.$$

The eigenvalues of H are the zeros of $z^p E_p(z)$. The eigenvalues of a $p \times p$ unitary Hessenberg matrix can be found with $O(p^2)$ operations. An algorithm exploiting connections with Szegő polynomials was given in [Gra86] and stabilized in [Gra99]. A different, stable algorithm involving matrix pencils was given in [BE91].

Examples:

1. Consider a stationary random signal with known autocorrelations $r_{xx}(0) = 1$ and $r_{xx}(1) = a < 1$. The other autocorrelations are assumed to be unknown. The Yule–Walker system

$$\begin{bmatrix} 1 & a \\ a & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -a_1(1) \end{bmatrix} = \begin{bmatrix} E_1 \\ 0 \end{bmatrix}$$

has solution $a_1(1) = a$ and $E_1 = 1 - a^2$. Thus, the maximum entropy estimate of the spectral density $S_x(e^{i\omega})$ is

$$S(e^{i\omega}) = \frac{1 - a^2}{|1 - ae^{-i\omega}|}.$$

64.7 Direction of Arrival Estimation

Definitions:

The **direction of arrival estimation** problem can be described as follows. We assume that n plane waves $s_j(k)$, $1 \leq j \leq n$ arrive at an array of m sensors from angles θ_j resulting in output $x_l(k)$, $1 \leq l \leq m$ from the sensors. We assume that the sensors lie in a plane and that it is necessary to estimate only a single angle of arrival for each signal. It is assumed that each sensor output is corrupted by noise $n_l(k)$, $1 \leq l \leq m$.

The sensors experience different delays and attenuations of the signal $s_j(k)$ depending on their position and the direction of arrival. If the signals are **narrow-band signals**

$$s_j(k) = \text{Re} \left(u_j(k) e^{i(\omega_0 k + v_j(k))} \right)$$

for slowly varying $u_j(k)$ and $v_j(k)$, then a signal delayed by l is approximately

$$s_j(k-l) = \text{Re} \left(u_j(k-l) e^{i(\omega_0 k + v_j(k-l))} e^{-i\omega_0 l} \right) \approx \text{Re} \left(u_j(k) e^{i(\omega_0 k + v_j(k))} e^{-i\omega_0 l} \right).$$

Thus, both delay and attenuation of the signals can be described by multiplication by a complex scalar dependent on θ_j . The observed signals are, therefore,

$$\mathbf{x}(k) = \sum_{j=1}^n \mathbf{a}(\theta_j) s_j(k) + \mathbf{n}(k),$$

where

$$\mathbf{x}(k) = \begin{bmatrix} x_1(k) \\ \vdots \\ x_m(k) \end{bmatrix}$$

and $\mathbf{a}(\theta)$ is the complex **array response vector**. In matrix notation the **signal model** is

$$\mathbf{x}(k) = A\mathbf{s}(k) + \mathbf{n}(k),$$

where

$$A = [\mathbf{a}(\theta_1) \quad \mathbf{a}(\theta_2) \quad \cdots \quad \mathbf{a}(\theta_n)].$$

The problem is to estimate the angles θ_j from a sequence $\mathbf{x}(k)$ of sensor output vectors. We assume that the components of the noise vector $\mathbf{n}(k)$ are uncorrelated, stationary, and zero mean. A more complete description of the problem is given in [RK89].

The **array manifold** is the set of all $\mathbf{a}(\theta)$ for θ in $[-\pi, \pi]$.

The **signal subspace** is the range space of the matrix A .

The **noise subspace** is the orthogonal complement of the signal subspace.

We define the spatial correlation matrices

$$R_{\mathbf{x}} = E[\mathbf{x}\mathbf{x}^*], \quad R_{\mathbf{s}} = E[\mathbf{s}\mathbf{s}^*], \quad \text{and} \quad R_{\mathbf{n}} = E[\mathbf{n}\mathbf{n}^*]$$

where we assume $R_{\mathbf{s}}$ is positive definite. These definitions apply directly when dealing with stationary random signals. In the deterministic case they can be replaced with time averages.

Facts:

1. If the signal and noise are uncorrelated, then

$$R_{\mathbf{x}} = AR_{\mathbf{s}}A^* + R_{\mathbf{n}}.$$

2. The signal model implies that except for the effect of noise the observation vectors $\mathbf{x}(k)$ lie in the signal subspace. Thus, in the absence of noise, any n linearly independent observation vectors $\mathbf{x}(k)$ will give a basis for the signal subspace. The directions θ_j are determined by the signal subspace so long as the mapping from the set of angles θ_j to the signal subspace is invertible. We assume that the sensor array is designed to ensure invertibility.

3. The signal subspace can be estimated by solving a generalized eigenvalue problem. The n eigenvectors associated with the n largest generalized eigenvalues of the pencil

$$R_x - \lambda R_n$$

are the basis for the estimated signal subspace. This method can be used to obtain matrices X_s and X_n such that $X_s X_s^*$ is a projection for the signal subspace and $X_n X_n^*$ is a projection for the noise subspace.

4. [Sch79] In order to obtain the angles θ_j the **MUSIC algorithm** searches the array manifold to find angles which are close to the signal subspace in the sense that

$$M(\theta) = \frac{\mathbf{a}^*(\theta)\mathbf{a}(\theta)}{\mathbf{a}^*(\theta)X_n X_n^* \mathbf{a}(\theta)}$$

is large. The values of θ for which $M(\theta)$ has a peak are taken as the estimates of the angles θ_j , $1 \leq j \leq n$. The basic approach of the MUSIC algorithm can be used for a variety of parameter estimation problems, including the problem of finding sinusoids in noise.

5. [RK89] The **ESPRIT** algorithm imposes a structure on the array of sensors. It is assumed that the array is divided into two identical subarrays displaced along an $\mathbb{R}^2$ vector Δ . Let $x_j(k)$ be the output of sensor j in the first subarray and $y_j(k)$ be the output of the corresponding sensor in the second subarray. The displacement Δ causes a delay in the signal $x_j(k)$ relative to $y_j(k)$. If the signals arriving at the array are again narrow band signals centered at frequency ω_0 , then the signal model is

$$\mathbf{x}(k) = \sum_{j=1}^n s_j(k) \mathbf{a}(\theta_j) + \mathbf{n}_x(k),$$

$$\mathbf{y}(k) = \sum_{j=1}^n s_j(k) e^{i\omega_0 \|\Delta\|_2 \sin(\theta_j)/c} \mathbf{a}(\theta_j) + \mathbf{n}_y(k),$$

where θ_k is the angle of arrival relative to the subarray displacement vector. The combined ESPRIT signal model including both subarrays is

$$\begin{bmatrix} \mathbf{x}(k) \\ \mathbf{y}(k) \end{bmatrix} = \begin{bmatrix} A \\ A\Phi \end{bmatrix} \mathbf{s}(k) + \begin{bmatrix} \mathbf{n}_x(k) \\ \mathbf{n}_y(k) \end{bmatrix},$$

where the diagonal matrix $\Phi = \text{diag}(e^{i\omega_0 \|\Delta\|_2 \sin(\theta_1)/c}, \dots, e^{i\omega_0 \|\Delta\|_2 \sin(\theta_n)/c})$ characterizes the delay between the subarrays.

6. To compute the θ_j using the ESPRIT algorithm, we start with the signal subspace associated with the ESPRIT signal model estimated as before using a generalized eigenvalue problem. We assume that a $2m \times n$ matrix S with columns spanning the signal subspace

$$S = \begin{bmatrix} S_x \\ S_y \end{bmatrix}$$

has been computed and compute the $2n \times 2n$ matrix

$$V = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix}$$